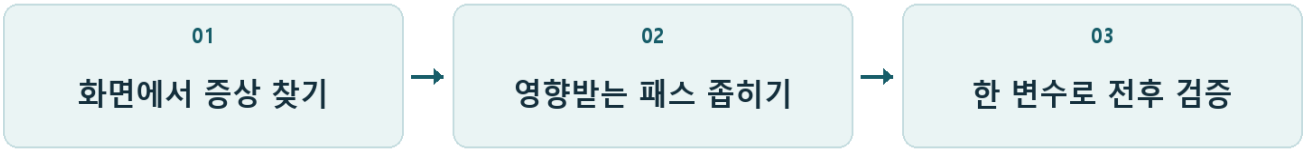
언리얼 엔진 5 렌더링 원리와 최적화

게임 배경 아티스트를 위한 기술 학습 시리즈

26개의 글로 렌더링의 흐름을 따라갑니다. 화면에서 발견한 문제를 깊이와 표면 데이터, 지오메트리, 조명, GPU 패스와 연결하고, 언리얼에서 원인을 하나씩 확인합니다.



실습 환경

Epic UE 5.8 문서의 데스크톱 경로를 기준으로 설명합니다. 프로젝트의 엔진 버전, RHI, Shader Model, 렌더러와 Scalability에 따라 기능과 메뉴, 패스 이름이 달라질 수 있습니다. Forward·모바일 등 별도 경로에 적용할 때는 기능 지원을 먼저 확인하세요.

비교 방법

간단한 복도나 골목 씬을 하나 만들어 벽, 반복 소품, 식생, 유리, 라이트를 차례로 추가하세요. 카메라·해상도·노출을 고정하고 실습마다 한 변수만 바꿉니다. 이미지 품질은 정지 화면과 이동 영상으로, 성능은 프레임과 관련 패스의 ms로 기록합니다.

그림은 렌더러의 핵심 관계를 보여 주는 개념도입니다. 실습 항목은 재현 절차이며, 수치 예시는 가정값입니다.

목차

렌더링 기초

01 렌더링 파이프라인 · 3
02 Forward와 Deferred · 13
03 GBuffer와 표면 정보 · 14
04 DBuffer와 데칼 · 15
05 Depth와 Early Z · 16
06 Overdraw와 화면 겹침 · 17

머티리얼과 빛

07 PBR과 재질 입력 · 18
08 BRDF와 반사 분포 · 19
09 Fresnel과 시선 각도 · 20
10 Shader와 순열과 WPO · 21
11 Substrate와 재질 레이어 · 22
12 Translucency와 굴절 · 23
13 SSS와 표면 아래 산란 · 24

지오메트리와 텍스처

14 Draw Call과 배칭 · 25
15 Instancing과 반복 배치 · 26
16 LOD와 화면 크기 · 27
17 Nanite와 가상화 지오메트리 · 28
18 Culling과 가시성 · 29
19 Mipmap과 텍스처 샘플링 · 30
20 Texture Streaming과 메모리 예산 · 31

라이팅과 화면 품질

21 Lumen과 동적 간접광 · 32
22 Virtual Shadow Maps와 페이지 캐시 · 33
23 TAA와 프레임 히스토리 · 34
24 TSR과 시간적 업스케일링 · 35

실전 성능 분석

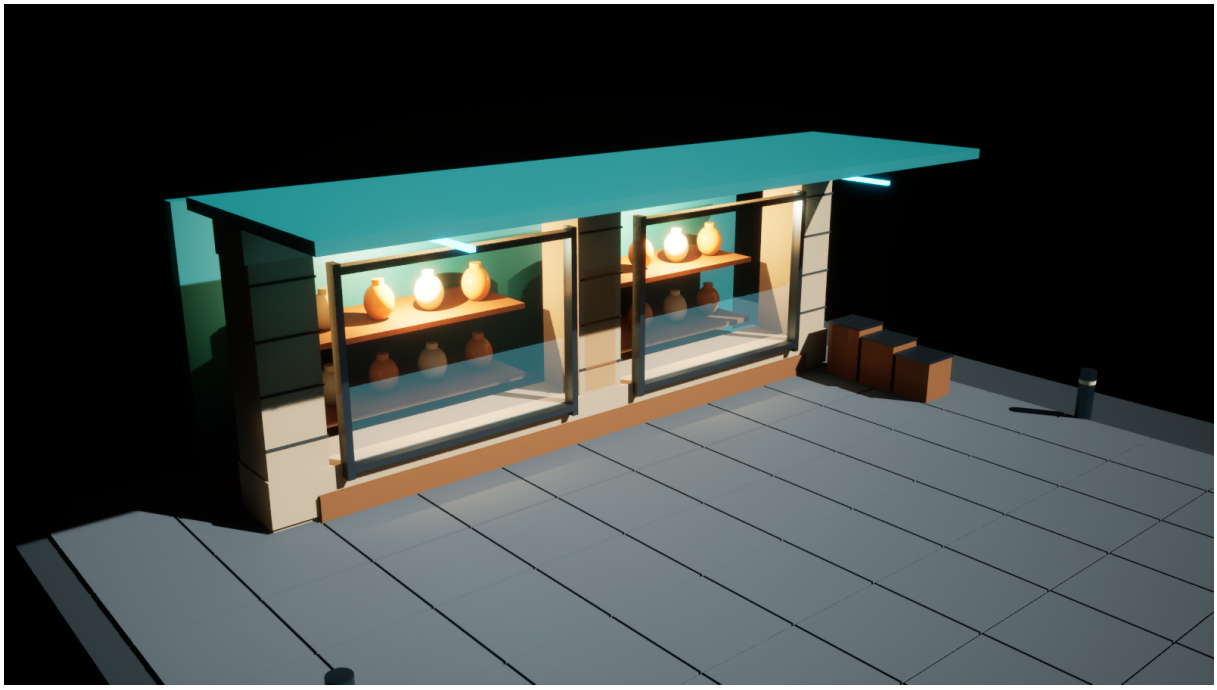
25 CPU와 GPU 병목 · 36
26 GPU Profiler와 전후 검증 · 37
보충 골목 씬의 성능 분석 · 38

출처 목록 · 39

01 렌더링 파이프라인

이 챕터의 목표 쇼윈도 하나를 보며 벽, 유리, 조명과 후처리가 화면에 기여하는 방식을 추적합니다. 같은 씬을 여러 진단 모드로 관찰하고 유리·데칼·Bloom을 하나씩 바꿔, 문제가 발생한 단계의 후보를 좁히는 것이 목표입니다.

직접 제작한 사례 실습 프로젝트의 CH01을 작은 상점 골목으로 확장했습니다. 포장석과 벽체, 금속 프레임, 선반과 도자기는 불투명 표면입니다. 두 쇼윈도에는 반투명 유리를, 중앙 석재 기둥에는 DBuffer 데칼을, 처마 아래에는 발광 바를 배치했습니다. 건축·재질·빛·합성이 동시에 필요한 배경 아트의 상황을 축소한 실습 장면입니다.

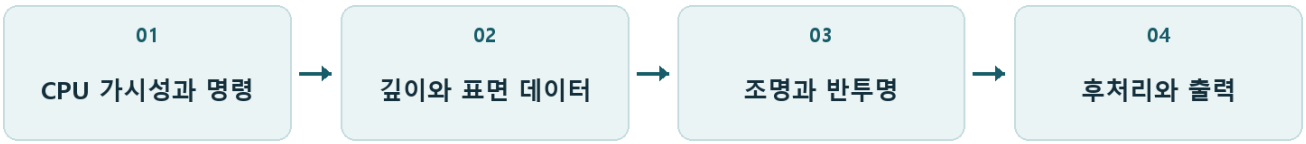


직접 제작·엔진 캡처. CH01 골목의 Lit 화면. 동일 카메라로 진단 화면과 A/B 결과를 비교합니다.

실습 레벨 /Game/RenderingLab/Chapters/Part01_RenderingBasics/CH01_RenderingPipeline/Maps/L_CH01_RenderingPipeline

읽는 순서 먼저 완성 화면의 물체를 식별한 뒤 파이프라인을 따라갑니다. 이후 Base Color·법선·깊이를 관찰하고, 실제로 유리와 후처리를 껐다 켜며 가설을 확인합니다.

한 프레임이 만들어지는 과정



개념도 / 실제 렌더 그래프에는 분기와 추가 패스가 있습니다.

1 CPU가 후보와 명령을 준비합니다 Game Thread는 씬 상태를 갱신하고 렌더 스레드와 RHI 경로는 그리기에 필요한 데이터를 준비합니다. 카메라 밖이거나 가려진 후보를 줄이고 메시 섹션·머티리얼·패스에 맞는 작업을 구성합니다. CPU와 GPU는 여러 프레임의 작업을 겹쳐 처리할 수 있습니다.

2 지오메트리를 화면 공간에 대응시킵니다 전통적인 래스터 경로에서 정점 셰이더는 정점의 위치를 변환하고, 래스터화는 삼각형이 덮는 화면 샘플과 보간값을 만듭니다. 픽셀 셰이더는 해당 위치에서 재질을 평가합니다. Nanite는 전용 가시성·래스터 경로를 사용하므로 모든 메시가 동일한 정점·픽셀 경로를 그대로 지난다고 일반화하지 않습니다.

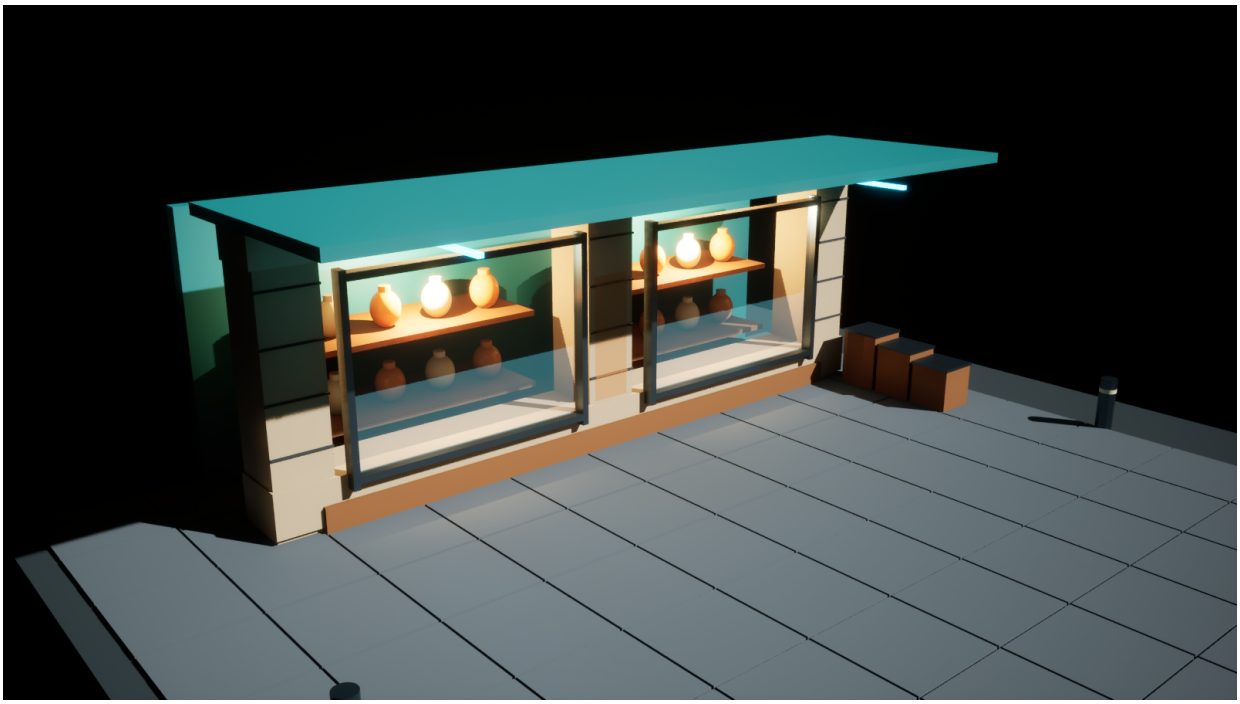
3 깊이와 표면 데이터를 준비합니다 구성에 따라 Depth Prepass가 깊이를 준비하고 DBuffer 데칼을 먼저 누적합니다. 불투명 Base Pass는 수신 데칼을 반영해 표면 정보를 기록합니다. GBuffer는 조명 평가에 필요한 재질 데이터이고 Scene Depth는 별도의 리소스입니다.

4 조명과 그림자를 평가합니다 빛의 방향, 표면의 법선과 반사 특성, 가려짐 등을 결합합니다. 직접광 외에 Lumen 간접광·반사도 최종 결과에 기여합니다. 그림자 맵 생성과 추적·캐시 갱신은 관련 작업 사이에 의존성이 있는 별도 경로입니다.

5 반투명과 후처리를 결합합니다 유리와 같은 표면은 불투명 배경 위에 별도의 셰이딩·합성을 요구합니다. 이후 Bloom, 톤매핑, 시간적 재구성 등 화면 작업이 이어집니다. 반투명의 종류에 따라 합성 시점은 DOF·업스케일러 등과의 관계가 달라집니다.

실제 실행 순서 읽기 위 흐름은 대표 데스크톱 Deferred 경로의 관계를 정리한 개념도입니다. 실제 프레임은 분기한 렌더 그래프이며 패스 이름과 순서는 버전·플랫폼·설정에 따라 달라집니다. 실행 순서를 검증할 때는 GPU 캡처의 이벤트와 리소스 의존성을 확인합니다.

씬에서 비교할 대상을 찾기



직접 엔진 캡처. 바닥·기둥, 두 쇼윈도와 내부 진열대, 처마 아래 발광 바를 관찰합니다.

1 벽과 바닥 01_OpaqueArchitecture 폴더에는 석재 기둥, 포장석과 금속 프레임이 있습니다. 같은 재질 색도 조명 방향과 표면 방향에 따라 Lit에서 다르게 보입니다. 포장석 틈은 실제 지오메트리이므로 Wireframe과 깊이에서도 확인할 수 있습니다.

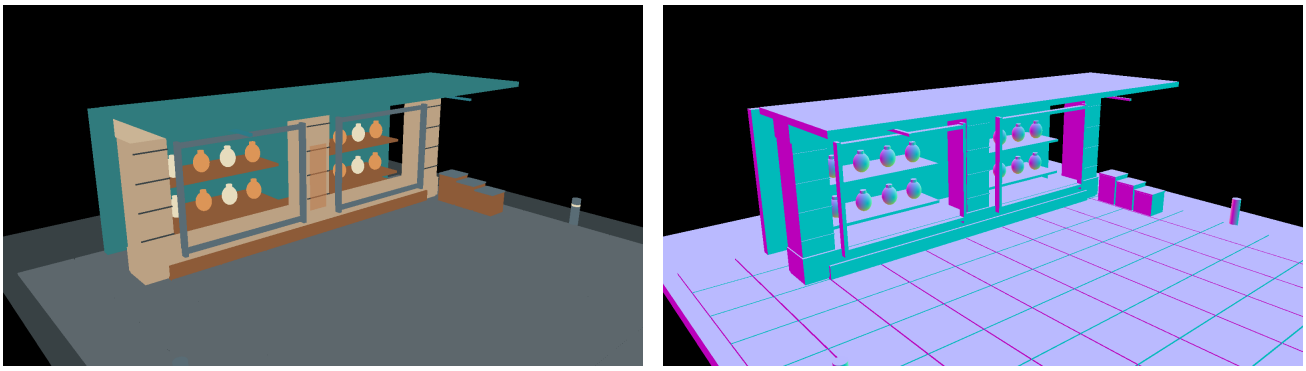
2 쇼윈도 내부와 유리 02_OpaqueDisplay의 선반·도자기 앞에 03_Translucency의 GlassPane_1과 GlassPane_2가 놓여 있습니다. 유리 두 개만 숨기면 내부 물체와 조명 배치를 유지한 채 반투명 합성의 영향을 관찰할 수 있습니다.

3 표면 얼룩과 발광 바 04_Decals의 WallPatina_DBuffer는 중앙 석재를 바꿉니다. 05_Signage의 발광 바는 Emissive 입력을 사용합니다. 빛나는 표면, 주변을 비추는 광원, 화면에서 퍼지는 Bloom은 각각 영향을 분리해 확인할 수 있습니다.

에디터에서 시작하기 CH01 맵을 열고 Outliner의 CH01_ComparisonCamera를 선택한 뒤 Pilot합니다. 06_Lighting과 08_PostProcess를 펼쳐 실습 대상을 확인합니다. 카메라와 수동 노출은 비교 동안 유지합니다. 에디터 구도와 실제 캡처 해상도는 별개이므로 비교 캡처는 모두 1600×900으로 맞춥니다.

Base Color와 법선을 분리해서 읽기

Base Color **World Normal**



직접 엔진 캡처 비교. 왼쪽 Base Color, 오른쪽 World Normal. 동일 장면의 다른 표면 데이터를 보여 줍니다.

Base Color에서 확인할 것 Viewport의 View Mode → Buffer Visualization → Base Color를 선택합니다. 석재와 도자기, 포장석의 입력 색이 어떻게 구분되는지 봅니다. Lit의 어두운 면이 이 화면에서도 어둡다면 재질 입력을, 이 화면에서는 비슷한데 Lit에서만 크게 다르면 조명·가림·반사 등을 다음 후보로 조사합니다.

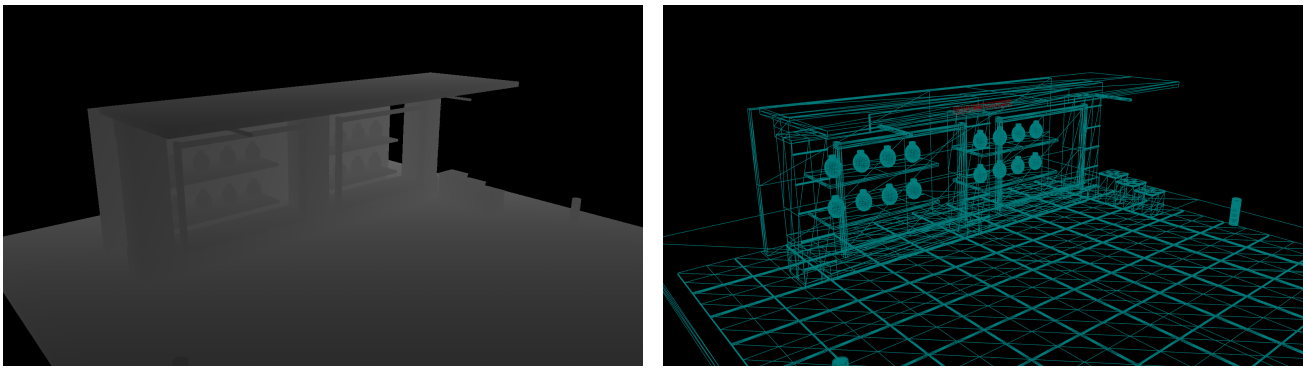
World Normal에서 확인할 것 World Normal을 선택합니다. 법선의 세 방향 성분을 색으로 표현하므로 방향이 다른 면은 색도 달라집니다. 구의 색이 연속적으로 변하는 것은 매끄럽게 보간된 표면 방향 때문입니다. 벽을 회전하면 월드 법선이 달라집니다. 노멀 맵이 없어도 서로 다른 색이 나타납니다.

직접 바꿔 보기 MI_CH01_Sandstone의 BaseColor만 바꾼 다음 두 버퍼를 다시 봅니다. 색 입력은 바뀌지만 메시와 법선을 바꾸지 않았으므로 World Normal은 같은 형태를 유지해야 합니다. 실습 뒤 원래 값으로 되돌립니다. 이 방식으로 바꾼 변수와 영향을 받은 데이터의 관계를 확인합니다.

버퍼 화면의 의미 이 이미지는 엔진의 진단 시각화입니다. 원시 GBuffer 채널을 그대로 덤프한 결과와는 표시 방식이 다를 수 있습니다. 특히 반투명 표면은 불투명 표면과 다른 경로로 처리되므로 Lit의 유리색이 모든 버퍼에 같은 방식으로 기록될 것이라 예상하지 않습니다.

깊이와 삼각형을 함께 관찰하기

Scene Depth **Wireframe**



직접 엔진 캡처 비교. 왼쪽 Scene Depth 진단 화면, 오른쪽 Wireframe. 색과 밝기는 표시용 표현입니다.

Scene Depth에서 확인할 것 Buffer Visualization → Scene Depth를 선택합니다. 가까운 볼라드, 앞쪽 포장석과 뒤쪽 진열대가 서로 다른 깊이로 표현되는지 확인합니다. 화면에 표시된 명암을 월드 거리 수치로 바로 읽지 않습니다. 하드웨어 깊이와 진단 화면의 변환 방식이 개입합니다.

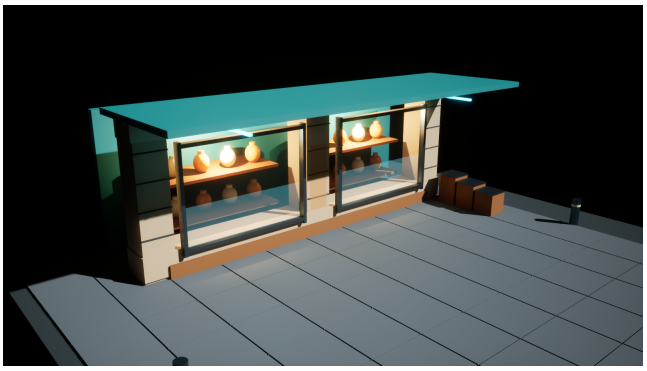
유리와 깊이의 관계 일반적인 반투명 유리는 불투명 표면처럼 기본 깊이를 덮어쓰지 않습니다. 따라서 기본 Scene Depth에서 유리 뒤 물체의 깊이가 관찰될 수 있습니다. Custom Depth나 특정 반투명 기능은 별도 설정이므로 이 실습의 기본 깊이와 구분합니다.

Wireframe에서 확인할 것 Alt+2 또는 View Mode → Wireframe을 선택해 삼각형 에지를 봅니다. 삼각형이 많아 보인다는 이유만으로 해당 장면의 병목을 정할 수는 없습니다. 표면이 화면을 덮는 크기, 셰이더, 그림자와 가시성도 영향을 줍니다. Alt+4로 Lit에 돌아갑니다.

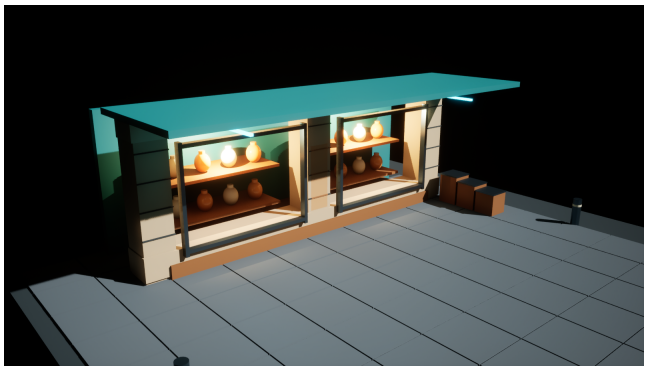
가려짐 실험 카메라 가까이에 불투명 큐브를 놓아 일부 도자기를 가립니다. Scene Depth가 앞쪽 큐브를 나타내는 위치와 Lit에서 가려지는 위치를 비교합니다. Early Z가 어떤 작업을 줄였는지까지 판단하려면 깊이 프리패스·가시성·GPU 이벤트를 추가로 확인해야 합니다.

유리를 숨겨 반투명 합성을 확인하기

유리 표시



유리 숨김



직접 엔진 A/B 캡처. 왼쪽 유리 표시, 오른쪽 GlassPane_1·2만 숨김. 카메라·조명·노출은 동일합니다.

실습 절차 Lit에서 원본 화면을 확인합니다. Outliner의 03_Translucency 폴더를 펼쳐 GlassPane_1과 GlassPane_2를 선택하고 에디터 가시성만 끕니다. 선반, 도자기, 프레임과 광원은 그대로 둡니다. 같은 카메라로 다시 캡처한 뒤 유리 표시 상태를 복원합니다.

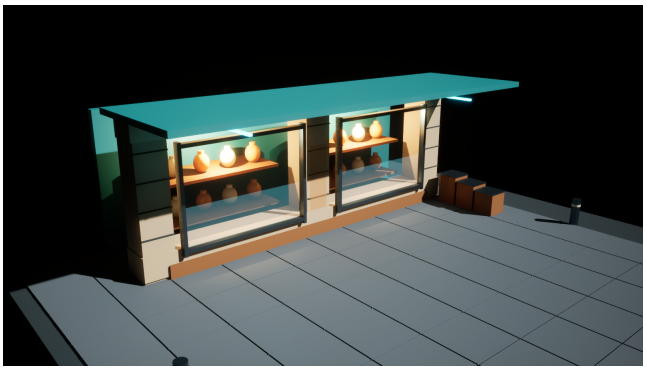
관찰 결과를 설명하기 두 이미지에서 쇼윈도 내부의 색과 대비, 유리 표면에 나타나는 반사 기여를 비교합니다. 이 실습의 유리는 Opacity 0.28, Roughness 0.10과 청록색 Base Color를 가진 단순 Translucent 재질입니다. 물리적 두께에 따른 흡수나 굴절을 정교하게 검증하는 실험은 이후 Translucency 챕터에서 다룹니다.

어느 단계를 조사할까 유리만 숨겼을 때 문제가 사라지면 반투명 셰이딩·정렬·합성 및 관련 반사를 조사할 근거가 됩니다. 불투명 배경이 동일하게 그려졌는지도 함께 확인합니다. Lumen처럼 다른 표면의 가시성과 화면 추적에 반응하는 기능 때문에 변화가 반투명 패스 하나에만 국한되지는 않을 수 있습니다.

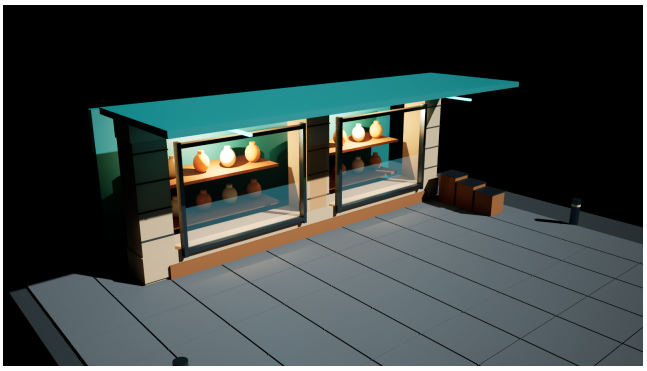
성능 검증으로 이어가기 stat gpu나 GPU 타임라인에서 Translucency 계열과 전체 GPU 시간을 기록합니다. A/B 순서를 바꿔 반복하고 워밍업 구간을 제외합니다. 이 문서의 비교 이미지는 시각적 변화의 증거이며, 고해상도 캡처 중의 프레임 시간을 성능 수치로 사용하지 않습니다.

Bloom과 발광을 구분하기

Bloom 1.0



Bloom 0.0



직접 엔진 A/B 캡처. 왼쪽 Bloom Intensity 1.0, 오른쪽 0.0. 발광 재질과 실제 광원은 유지했습니다.

실습 절차 08_PostProcess의 CH01_FixedExposure를 선택합니다. Bloom Intensity의 Override를 유지한 채 값을 1.0에서 0.0으로 바꿉니다. 카메라 노출, M_CH01_SignEmission과 진열장 Point Light는 수정하지 않습니다. 캡처 후 값을 1.0으로 되돌립니다.

관찰할 차이 처마 아래 밝은 바의 주변으로 퍼지는 화면 효과를 비교합니다. Bloom을 줄여도 발광 재질의 밝은 중심 자체는 남습니다. 포스트 프로세스가 만드는 확산과 장면 조명의 기여를 따로 생각하면, 간판이 과하게 번질 때 어떤 값을 먼저 바꿀지 판단하기 쉽습니다.

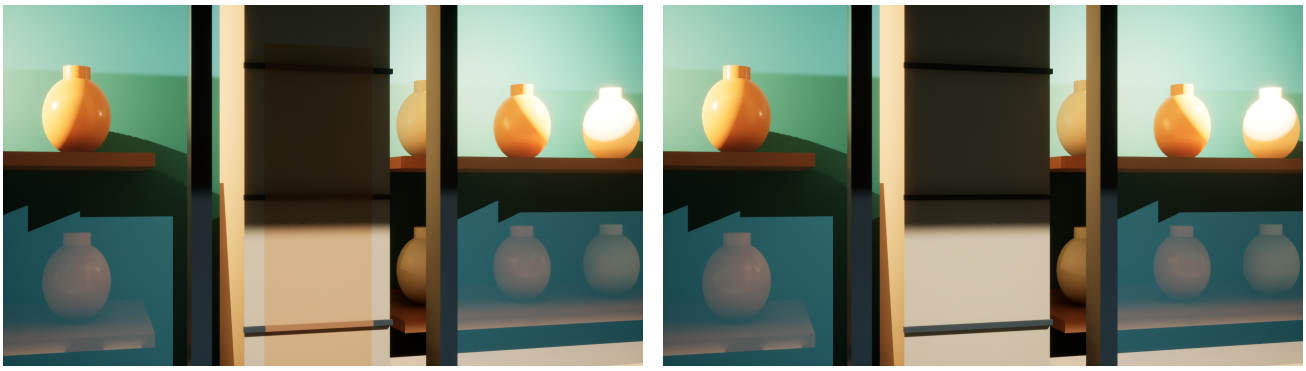
발광이 주변에 미치는 영향 이 씬은 Lumen을 사용하므로 Emissive가 간접광에 기여할 수 있습니다. 진열장에는 별도의 WarmDisplayLight도 있습니다. Bloom만 바꾸는 실험에서 두 광원의 에너지를 동시에 바꾸지 않아야 비교 의도가 유지됩니다.

아트 실무의 진단 밝은 간판을 고치려다 Base Color를 어둡게 만들기 전에 Bloom, 수동 노출과 Emissive 값을 각각 확인합니다. 톤매핑으로 높은 밝기가 압축되는 현상도 포함해 판단합니다. 최종 출력 색은 Base Color 값만으로 결정되지 않습니다.

데칼과 조명에서 생긴 변화를 찾기

데칼 표시 근접

데칼 숨김 근접



직접 엔진 A/B 근접 캡처. 같은 중앙 기둥에서 데칼 표시와 숨김을 비교합니다.

데칼 실험 04_Decals의 WallPatina_DBuffer만 숨기고 원본 Lit와 중앙 기둥을 비교합니다. 이 데칼은 색과 Roughness 계열 표면 값을 수정합니다. DBuffer 경로에서는 Base Pass 전에 데칼 정보를 준비하고 수신 머티리얼이 이를 적용합니다. 데칼이 최종 색을 화면 위에 단순히 붙이는 것처럼만 이해하면 반사 변화의 원인을 놓치기 쉽습니다.

조명 실험 데칼을 다시 표시하고 View Mode → Lighting Only를 선택합니다. 표면의 색을 중립 재질로 바꾸어 빛과 음영 구조를 관찰합니다. 이 화면은 최종 Lit의 밝기를 정확히 분해한 수치가 아니라 조명 상태를 읽기 위한 진단 표현입니다.

같은 위치를 여러 데이터로 확인하기 중앙 기둥이 어둡다면 Base Color, World Normal, Lighting Only, 데칼 가시성을 차례로 확인합니다. 입력 색, 방향, 빛과 표면 수정 중 어떤 변화와 연결되는지 기록합니다. 서로 다른 진단 모드의 픽셀 밝기를 그대로 빼서 조명 에너지라고 해석하지 않습니다.

실무 예시 모듈 벽면의 얼룩은 데칼로 변주하면서 기본 재질은 공유할 수 있습니다. 투영 박스를 좁히고 겹침을 줄였을 때 시각적 범위가 유지되는지 확인하고, 비용은 별도의 전후 프로파일링으로 확인합니다.

제작 과정을 직접 재현하기

1 기본 씬 열기 RenderingLab.uproject를 UE 5.8.4에서 엽니다. CH01 레벨과 CH01_ComparisonCamera를 선택합니다. CH01은 Deferred, Lumen, VSM, TSR과 Substrate Blendable 시작 설정을 사용합니다. 이 비교를 위해 렌더러 설정을 바꿀 필요는 없습니다.

2 불투명 재질 만들기 CH01의 Materials 폴더에서 MI_CH01_Sandstone과 MI_CH01_MetalFrame을 엽니다. 부모 M_LabSurface는 BaseColor, Metallic, Roughness 파라미터를 연결한 기본 표면입니다. 금속 프레임은 Metallic 0.85, Roughness 0.26이고 석재는 Metallic 0, Roughness 0.82입니다. 학습용 수치이며 측정된 실물 재질 데이터는 아닙니다.

3 유리와 발광 구성하기 M_CH01_Glass는 Translucent, Surface ForwardShading을 사용합니다. Base Color (0.10, 0.32, 0.38), Opacity 0.28, Roughness 0.10을 연결합니다. M_CH01_SignEmission의 Emissive 입력은 (0.04, 3.5, 5.0)입니다. 표시한 벡터 값은 선형 재질 입력이며 문서 이미지의 sRGB 픽셀 값과 일치하지 않습니다.

4 빛과 비교 조건 맞추기 Sun_Key와 WarmDisplayLight_1·2가 직접광을 만듭니다. CH01_FixedExposure는 Manual 노출과 물리 카메라 노출 비적용을 사용합니다. 비교 중 라이트와 노출을 고정합니다. 캡처 도구는 Screen Percentage 100, Dynamic Resolution 꺼짐과 1600×900 출력을 사용합니다.

5 캡처 재현하기 File → Execute Python Script에서 Content/Python/capture_ch01.py를 실행하면 이 챕터의 진단·A/B 이미지를 순서대로 저장합니다. 이 도구는 전용 에디터 세션에서 실행하고, 실행 전 다른 작업을 저장합니다. 완료 시 에디터가 종료됩니다. 원본과 같은 파일 이름으로 캡처를 갱신합니다.

생성 도구의 범위 build_ch01.py는 CH01 레벨의 액터를 재구성하는 제작 도구입니다. 기존 CH01을 수정한 뒤 다시 실행하려면 변경을 커밋하거나 복사본을 확보합니다. 일반적인 학습과 재캡처에는 이미 저장된 레벨과 capture_ch01.py를 사용합니다.

검증 기록과 다음 실습

이 챕터에서 확인한 것 UE 5.8.4와 NVIDIA GeForce RTX 3070 Ti의 DX12 에디터에서 CH01을 렌더링했습니다. Lit, Base Color, World Normal, Scene Depth, Roughness, Wireframe, Lighting Only와 유리·Bloom·데칼 A/B 및 근접 화면을 촬영했습니다. 원본 이미지 12개와 CaptureManifest.json은 Docs/Images/CH01에 있습니다.

관찰과 측정을 구분하기 본 실습은 각 기능이 화면에 미치는 영향을 확인합니다. 처리 비용을 결론내리려면 대상 기기에서 워밍업한 동일 카메라 경로를 반복 실행해야 합니다. 고해상도 캡처는 렌더링을 동기화하고 작업량도 바꿀 수 있으므로 이 챕터에 임의의 ms 개선값을 붙이지 않았습니다.

검증 체크 ① Base Color만 바꿨을 때 World Normal의 형태가 유지되는가. ② 유리만 숨겼을 때 내부 선반과 도자기는 남는가. ③ Bloom만 껐을 때 발광 바의 중심은 남는가. ④ 데칼을 숨겼을 때 중앙 기둥의 표면 변화가 사라지는가. ⑤ 비교마다 카메라·해상도·노출을 유지했는가.

문제가 생기면 장면이 검게 보이면 셰이더·거리장 준비가 끝났는지, Sun_Key와 노출이 유효한지 확인합니다. 유리가 불투명하면 올바른 머티리얼과 Blend Mode를 봅니다. 진단 모드 뒤에는 Alt+4로 Lit에 돌아갑니다. 캡처 파일이 누락되면 Saved/Logs/CH01Capture.log와 CaptureManifest.json의 complete 값을 확인합니다.

핵심 정리 메시 배치, 표면 입력, 조명, 반투명과 후처리는 서로 다른 데이터를 바꾸면서 최종 화면에 연결됩니다. 증상이 보이는 위치를 여러 진단 모드와 한 변수의 전후 화면으로 확인하면 조사할 패스와 설정을 좁힐 수 있습니다. 다음 CH02에서는 표면과 조명을 평가하는 시점이 다른 Forward와 Deferred를 비교합니다.

Epic 공식 자료

Viewport Modes

Decal Materials

Lit Translucency

Render Dependency Graph

Scripting the Editor Using Python

02 Forward와 Deferred

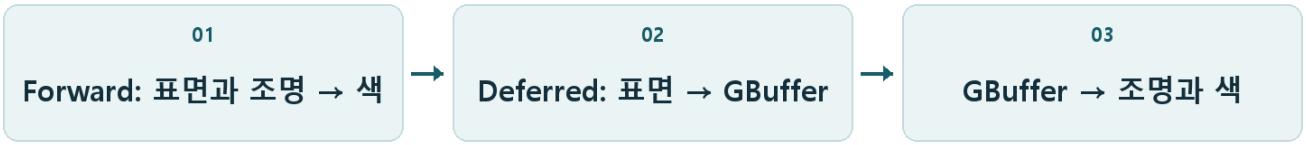


그림 02 / Forward와 Deferred 개념도

원리 Forward는 불투명 표면을 그리는 과정에서 영향을 주는 광원을 평가한다. Deferred는 먼저 표면 정보를 저장하고 조명 단계에서 그 정보를 읽는다. 여기서는 데스크톱 경로를 비교한다.

작동 과정 UE의 Forward Renderer는 광원과 Reflection Capture를 카메라 절두체 공간의 그리드로 선별한다. Deferred에서도 광원의 영향 범위를 제한한다. 같은 광원 수라도 여러 광원의 영향 범위가 화면에서 겹치면 평가할 픽셀이 늘어난다.

아트 실무 VR 전시장에서는 MSAA와 광원·반사 기능 요구를 먼저 확인한다. 동적 간접광을 사용하는 데스크톱 골목에서는 Lumen과 렌더러의 호환성을 먼저 확인한다.

성능 비용 Deferred는 화면 크기의 중간 표면 데이터에 대한 읽기·쓰기 비용을 낸다. Forward의 이점은 장면과 기능 선택에 따라 달라지며 그림자, 반사, AA 선택도 전체 비용을 바꾼다.

언리얼 실습 프로젝트 복제본에서 Project Settings → Rendering → Forward Shading을 바꾸고 재시작한다. ① 두 경로가 공통 지원하는 조명·반사·AA를 맞춘다. ② 겹치는 라이트 수를 1개와 여러 개로 바꾼다. ③ 동일 화면에서 Base Pass와 조명 그룹 및 총 GPU ms를 기록한다.

핵심 요약 조명을 언제 평가하는지와 필요한 기능의 지원 여부를 함께 비교한다.

Epic 공식 자료

[01] Rendering Settings
[04] Forward Shading Renderer

03 GBuffer와 표면 정보

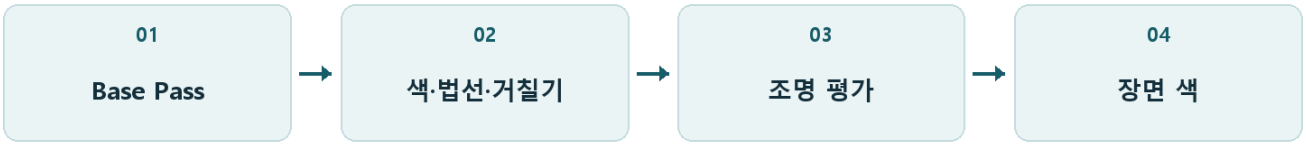


그림 03 / GBuffer와 표면 정보 개념도

원리 GBuffer는 Deferred 조명이 읽는 표면 데이터 집합이다. Base Color, 법선, Roughness 등은 조명에 대한 표면 반응을 결정한다. Scene Depth는 함께 참조하는 별도의 깊이 리소스다.

작동 과정 Base Pass가 가시 불투명 표면의 정보를 기록하면 조명 단계가 법선과 재질 파라미터를 읽고, 필요에 따라 깊이와 카메라 정보로 표면 위치를 재구성한다. 실제 패킹과 데이터 형식은 렌더러·플랫폼·Substrate 설정에 따라 달라진다.

아트 실무 벽이 젖은 듯 보일 때 Roughness와 World Normal을 따로 본다. 거칠기는 정상인데 하이라이트 방향이 어긋난다면 노멀 맵이나 탄젠트부터 검사한다.

성능 비용 가상 데이터가 32 B/px라면 1080p는 약 63.3 MiB, 3840×2160은 약 253.1 MiB다. 이는 픽셀 수에 따른 저장량 예시다. 실제 대역폭은 쓰기·읽기 횟수와 압축·캐시에도 좌우되며, 고정 형식에서 노드가 늘어도 바이트 수는 그대로일 수 있다.

언리얼 실습 ① 같은 벽을 Roughness 0.2와 0.8 두 인스턴스로 나눈다. ② View Mode → Buffer Visualization에서 Roughness와 Base Color를 비교한다. ③ GPU 캡처에서 Base Pass의 실제 타깃을 확인하고 내부 해상도를 바꿔 시간 변화를 기록한다.

핵심 요약 버퍼 내용, 저장 형식, 셰이더 계산량을 각각 확인한다.

Epic 공식 자료

[02] Render Dependency Graph
[05] Programming with Substrate GBuffer Formats

04 DBuffer와 데칼

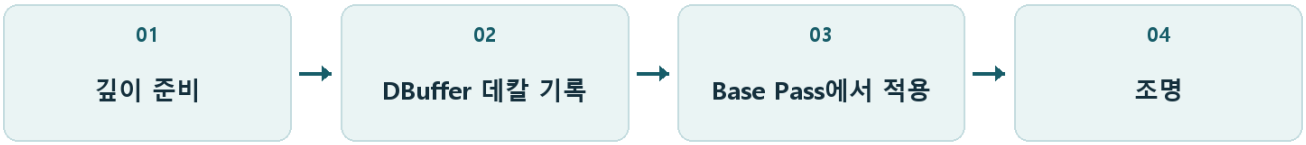


그림 04 / DBuffer와 데칼 개념도

원리 DBuffer는 투영 데칼의 표면 수정 값을 모으는 중간 데이터다. Base Pass의 수신 머티리얼이 이를 읽어 기존 표면 값에 반영한다.

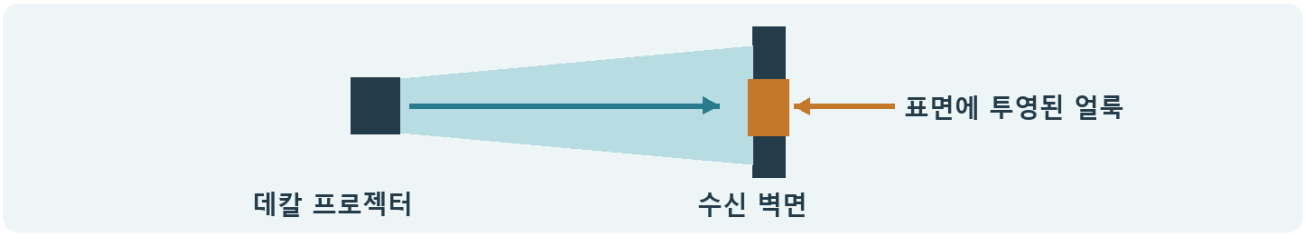
작동 과정 프로젝터 볼륨 안에서 씬 깊이로 수신 표면의 위치를 구해 데칼을 샘플한다. DBuffer는 색·법선·거칠기 계열을 전달한다. 별도의 GBuffer 데칼 경로는 Base Pass 뒤, 조명 전에 기존 GBuffer에 합성한다. 경로는 플랫폼과 프로젝트 구성에 따라 달라진다.

아트 실무 모듈 벽면에 빗물 자국을 붙일 때 투영 박스를 벽 두께에 맞춘다. 벽 뒤 소품까지 묻는다면 볼륨 깊이, Receives Decals, 수신 머티리얼의 Decal Response를 확인한다.

성능 비용 화면을 크게 덮는 투영 박스와 중첩이 데칼 작업량을 늘린다. 깊이 프리패스 및 수신 머티리얼 비용도 관련된다. 이미 다른 기능이 프리패스를 요구하면 DBuffer만의 추가 비용을 따로 측정해야 한다.

언리얼 실습 ① Deferred Decal 도메인의 얼룩을 벽에 투영한다. ② 데칼 1개와 겹친 8개를 비교한다. ③ 같은 얼룩이 보이도록 박스만 줄여 데칼 패스와 Base Pass를 다시 잰다. DBuffer 표현식으로 직접 합성할 때는 기본 Decal Response와 중복 적용 여부를 본다.

핵심 요약 데칼 개수와 함께 투영 면적, 겹침, 수신 설정을 본다.



단면도 / 투영 볼륨이 벽과 만나는 영역에서 데칼을 평가합니다.

Epic 공식 자료

[05] Programming with Substrate GBuffer Formats
[06] Decal Materials

05 Depth와 Early Z

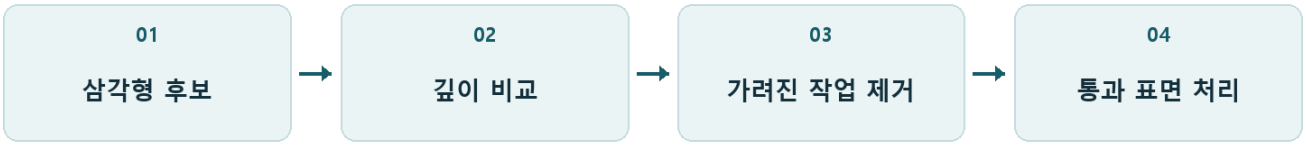


그림 05 / Depth와 Early Z 개념도

원리 깊이 버퍼는 화면 위치별 깊이를 저장한다. Early Z는 가능한 경우 비싼 픽셀 셰이딩 전에 가려진 프래그먼트를 거르는 GPU 동작이다.

작동 과정 Depth Prepass가 먼저 깊이를 채우면 뒤의 패스가 그 값을 이용할 수 있다. 프리패스 구성, 그리기 순서와 머티리얼의 깊이·마스크 동작에 따라 절약량이 달라진다. Scene Depth는 투영된 값이므로 일반적인 월드 거리와 같은 값으로 해석하면 안 된다.

아트 실무 계단 뒤 장식처럼 대부분 가려지는 불투명 디테일과 잎사귀 카드처럼 구멍이 많은 실루엣을 구분한다. 화면을 덮는 면적이 같아도 깊이에서 제거할 수 있는 작업량이 다르다.

성능 비용 프리패스에는 지오메트리 제출과 깊이 쓰기 비용이 든다. Masked 표면은 마스크를 평가해야 커버리지를 확정할 수 있다. 픽셀 절약량과 프리패스 비용을 함께 본다.

언리얼 실습 ① 불투명 벽 뒤에 복잡한 머티리얼의 큰 패널을 놓는다. ② 가림판이 패널을 완전히 가리는 경우와 일부 드러내는 경우를 비교한다. ③ Scene Depth와 PrePass/Base Pass 시간을 기록한다. 오브젝트 컬링의 영향은 일부가 계속 보이는 조건으로 줄인다.

핵심 요약 깊이 프리패스는 픽셀 작업을 줄일 수 있는 선행 투자다.

Epic 공식 자료

[02] Render Dependency Graph
[03] Introduction to Performance Profiling and Configuration
[06] Decal Materials
[30] Visibility and Occlusion Culling

06 Overdraw와 화면 겹침

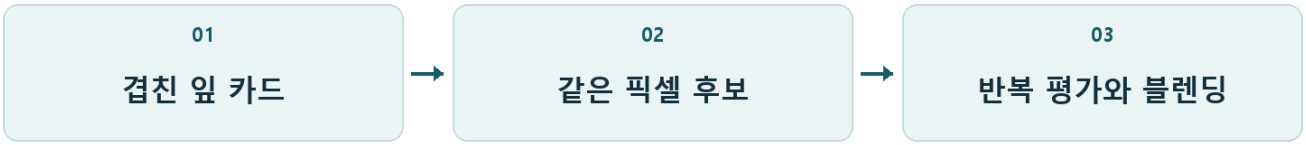


그림 06 / Overdraw와 화면 겹침 개념도

원리 Overdraw는 같은 화면 위치에 여러 프래그먼트가 도달해 작업이 반복되는 현상이다. 불투명 표면, 알파 마스크, 반투명 레이어의 깊이 처리 방식이 서로 다르다.

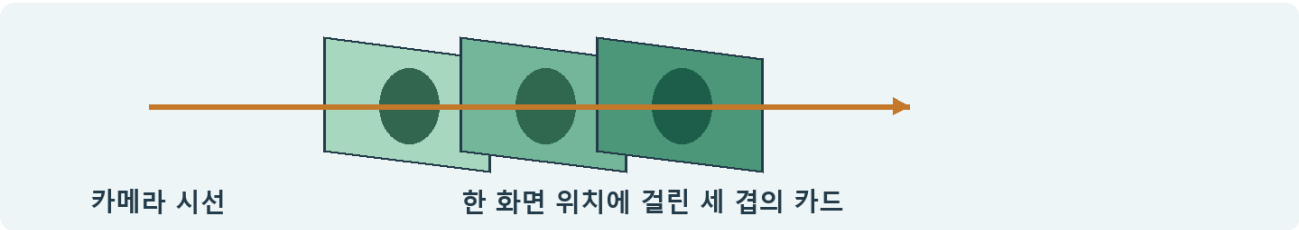
작동 과정 Opaque는 가려진 표면을 깊이로 걸러낼 수 있다. Masked는 텍스처를 평가해 픽셀을 버릴 수 있다. Translucent는 보통 불투명 깊이에 의해 가려질 수 있으나 앞뒤 투명층끼리는 합성해야 한다.

아트 실무 큰 사각형에 작은 잎만 그린 카드를 여러 겹 쌓으면 숲 가장자리가 비싸질 수 있다. 실루엣에 맞게 카드를 다듬거나 레이어 수를 줄이고, 실루엣 변화와 그림자를 함께 본다.

성능 비용 화면 점유율과 겹침, 픽셀당 셰이더 비용이 동시에 커지면 영향이 커진다. Shader Complexity의 색은 진단 신호이며 실제 ms나 메모리 대역폭을 직접 표시하지 않는다.

언리얼 실습 ① 같은 잎 텍스처를 큰 카드와 윤곽을 다듬은 카드에 적용한다. ② 근거리·원거리에서 Shader Complexity 및 지원되는 Quad Overdraw를 본다. ③ 삼각형 수 변화와 GPU ms를 기록한다. Nanite 메시는 Nanite Overdraw도 별도로 확인한다.

핵심 요약 삼각형을 조금 늘려도 불필요한 픽셀 커버리지를 줄이면 이득일 수 있다.



측면도 / 카메라 한 시선에 여러 카드가 걸릴 수 있습니다.

Epic 공식 자료

[02] Render Dependency Graph
[07] Viewport Modes
[19] Nanite Virtualized Geometry

07 PBR과 재질 입력

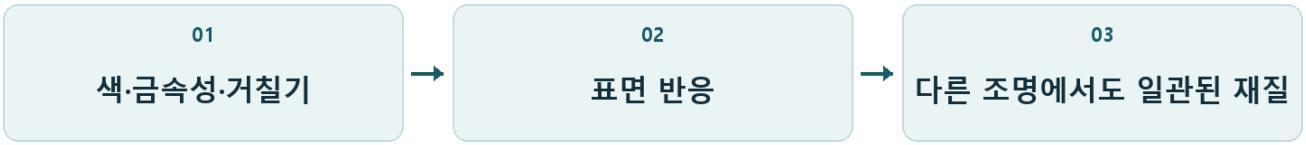


그림 07 / PBR과 재질 입력 개념도

원리 PBR은 재질과 빛의 상호작용을 물리적으로 그럴듯하게 근사하는 제작 방식이다. Base Color, Metallic, Roughness, Specular는 다른 역할을 한다.

작동 과정 일반적인 금속성 워크플로에서 비금속의 Base Color는 확산 반사 색, 금속의 Base Color는 주로 반사 색에 관여한다. Roughness를 높이면 반사가 넓게 퍼진다. 순수 재료는 Metallic 0 또는 1을 기준으로 두고, 경계의 혼합이나 픽셀보다 작은 패턴의 평균에는 중간값이 나올 수 있다.

아트 실무 페인트가 벗겨진 철문에서 페인트와 녹을 비금속, 노출 금속을 금속으로 분리한다. 방향광 그림자를 Base Color에 직접 칠하면 조명이 바뀔 때 방향이 충돌한다.

성능 비용 거칠기 수치 자체를 바꾸는 일과 추가 텍스처·복잡한 블렌딩을 넣는 일의 비용은 다르다. ORM 채널 패킹은 호환되는 샘플 조건에서 읽기를 줄일 수 있으며 데이터 맵은 선형으로 읽는다.

언리얼 실습 ① Material Instance에서 Metallic 0/1과 Roughness 0.15/0.65 조합을 만든다. ② Skylight와 넓은 반사광에서 비교한다. ③ Buffer Visualization으로 입력을 확인하고 광원 방향을 바꿔 인공적으로 구워 넣은 하이라이트가 있는지 본다.

핵심 요약 재질의 속성은 재질 입력에, 장면의 조명 변화는 조명에 맡긴다.

Epic 공식 자료

[08] Physically Based Materials
[09] Using Fresnel

08 BRDF와 반사 분포

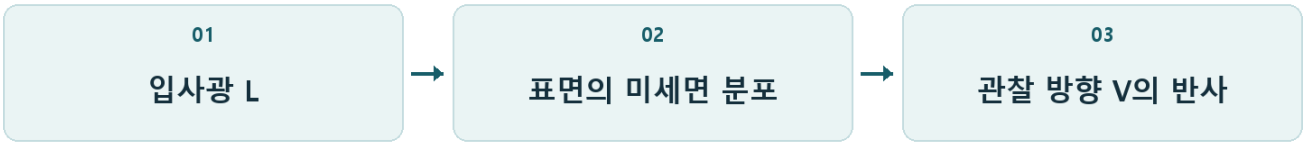


그림 08 / BRDF와 반사 분포 개념도

원리 BRDF는 입사 방향에서 들어온 빛이 관찰 방향으로 얼마나 반사되는지를 기술하는 함수다. BSDF는 표면을 통한 투과까지 포괄하는 더 넓은 개념이다.

작동 과정 대표적인 미세면 모델의 스펙큘러 항은 D×F×G를 4(N·L)(N·V)로 나눈 형태다. D는 반사에 맞는 미세면 방향의 분포, F는 각도별 반사율, G는 미세면 사이의 가림을 근사한다. 실제 화면 밝기에는 입사광과 입사각, 그림자 등이 더 관여한다.

아트 실무 돌 바닥의 닳은 길과 거친 가장자리에 서로 다른 Roughness를 준다. 같은 Base Color로도 반사 폭의 차이가 재질 사용감을 전달한다.

성능 비용 Roughness 값만 조절해도 같은 셰이더일 수 있다. 추가 코팅 로브나 복수 closure, 고가의 반사 경로가 들어가면 평가할 작업이 달라진다.

언리얼 실습 ① 같은 구와 동일 조명에서 Roughness 세 단계를 만든다. ② Rect Light의 크기는 고정하고 반사 폭을 비교한다. ③ 다음에는 재질을 고정하고 라이트 크기만 바꿔 두 변수의 영향을 분리한다.

핵심 요약 하이라이트 모양은 재질과 광원의 크기·방향이 함께 만든다.

Epic 공식 자료

[08] Physically Based Materials
[10] Overview of Substrate Materials
[31] Real Shading in Unreal Engine 4 · Brian Karis

09 Fresnel과 시선 각도

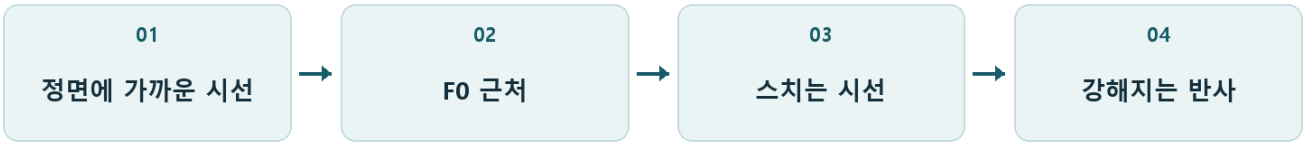


그림 09 / Fresnel과 시선 각도 개념도

원리 Fresnel은 계면의 반사율이 각도에 따라 달라지는 성질이다. F0는 법선 방향 입사의 반사율이다. 일반적인 유전체는 스치는 각도에서 반사율이 커진다.

작동 과정 Schlick 근사에서는 F0와 스치는 각도의 반사값 사이를 각도 함수로 보간한다. 미세면 BRDF에서는 시선과 미세면 반벡터 사이의 각도를 사용한다. UE의 Fresnel 머티리얼 노드는 화면 테두리 마스크에도 쓰이는 표현 도구다.

아트 실무 바닥의 얕은 물기는 내려다볼 때보다 복도를 낮게 바라볼 때 반사가 강하다. 발광 테두리를 넣으면 빛이 없는 곳에서도 밝아지므로 자연 반사와 효과 마스크를 별개로 설계한다.

성능 비용 마스크 계산 자체보다 마스크가 추가하는 샘플, 발광·반투명 레이어, 반사 기능이 더 큰 측정 대상일 수 있다.

언리얼 실습 ① 반사 환경이 있는 레벨에서 물기 있는 판을 정면과 낮은 시선에서 본다. ② 별도 재질에서 Fresnel 노드 출력을 Emissive에 연결해 마스크만 확인한다. ③ 두 결과를 같은 구도에서 비교한다.

핵심 요약 F0는 정면의 반사율이며 Fresnel 노드로 만든 림은 별도의 아트 제어다.

Epic 공식 자료

[08] Physically Based Materials
[09] Using Fresnel
[10] Overview of Substrate Materials
[31] Real Shading in Unreal Engine 4 · Brian Karis

10 Shader와 순열과 WPO

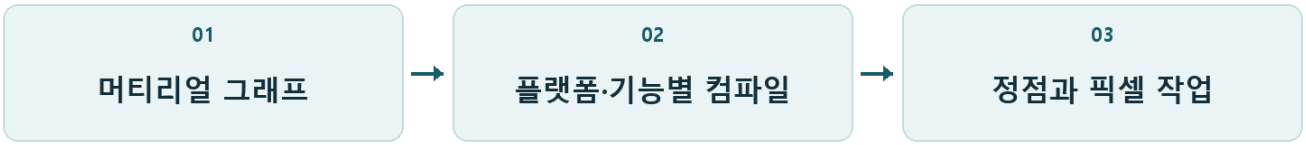


그림 10 / Shader와 순열과 WPO 개념도

원리 셰이더는 GPU가 실행하는 프로그램이다. UE 머티리얼 그래프는 여러 패스와 플랫폼에 맞춰 컴파일된다. Static Switch 조합은 실제 사용하는 구성에 따라 셰이더 순열을 늘릴 수 있다.

작동 과정 정점 경로의 WPO는 메시 정점 위치를 움직인다. 픽셀 경로는 각 표면 위치에서 텍스처나 반사 파라미터를 평가한다. 정적 스위치는 컴파일 시 경로를 선택하고 일반 Scalar Parameter는 같은 순열의 실행 값을 바꾼다.

아트 실무 풀의 흔들림을 WPO로 만들 때 근거리와 원거리 변형 요구가 다르다. 그림자도 변형을 따라가는지, 변형한 정점이 기존 bounds 밖으로 나가는지 함께 본다.

성능 비용 ALU, 텍스처 샘플과 캐시, 픽셀 커버리지, 정점 밀도, WPO에 따른 그림자 무효화가 다른 비용을 만든다. 컴파일 및 쿠킹 시간은 실행 GPU ms와 별도다.

언리얼 실습 ① 같은 풀 메시로 WPO 켜짐/꺼짐 재질을 만든다. ② Material Editor의 Platform Stats에서 대상 셰이더 모델을 비교한다. ③ 같은 밀도로 배치해 Base Pass와 Shadow Depths를 측정하고 움직임 범위를 확인한다.

핵심 요약 그래프 노드 수, 생성된 순열 수, 실제 프레임 비용을 따로 읽는다.

Epic 공식 자료

[02] Render Dependency Graph
[11] Material Editor UI
[12] Material Inputs

11 Substrate와 재질 레이어

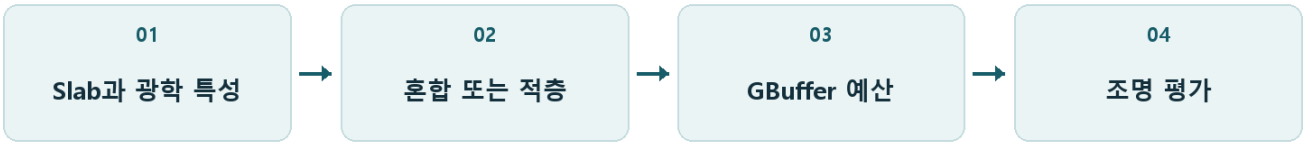


그림 11 / Substrate와 재질 레이어 개념도

원리 Substrate는 Slab BSDF 같은 단위로 재질의 광학적 성질을 구성한다. Slab의 Diffuse Albedo, F0, Roughness와 산란 파라미터를 조합해 복합 표면을 만든다.

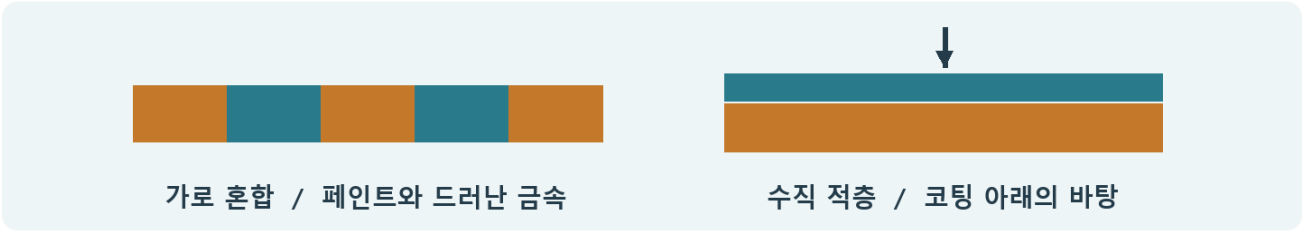
작동 과정 Horizontal Blend는 표면에서 서로 다른 재질의 비중을 혼합하고 Vertical Layer는 위층을 통과하는 빛과 아래층 반응을 표현한다. 파라미터 블렌딩은 여러 평가를 하나의 근사로 줄일 수 있다. Adaptive는 closure 수·픽셀당 바이트 예산에 맞춰 단순화될 수 있다.

아트 실무 벗겨진 철문은 페인트와 철의 가로 혼합, 물막이 있는 돌은 위층과 바탕의 적층을 후보로 둔다. 넓은 바닥에서는 거칠기와 법선만 바꾼 단순 버전도 만들어 시각 차이를 비교한다.

성능 비용 Blendable은 고정 메모리와 예측 가능한 비용을 지향한다. Adaptive 비용은 화면에 보이는 재질 복잡도와 플랫폼에 의존한다. 60Hz 지향은 설계 목표이며 프로젝트의 FPS는 직접 측정한다. 5.8 문서 기준 Adaptive는 지원 플랫폼에서 DBuffer를 강제한다.

언리얼 실습 ① 프로젝트에서 Substrate와 GBuffer Format 설정을 확인하고 기존 재질은 복제한다. ② 단일 Slab, 2층, 파라미터 혼합 세 버전을 만든다. ③ Substrate 시각화와 Material Editor의 단순화 정보를 보고 화면 점유율을 고정해 패스 시간을 비교한다.

핵심 요약 그래프의 층 수와 실행 시 평가되는 closure 수는 달라질 수 있다.



단면도 / 표면을 나눠 섞는 방식과 위아래로 쌓는 방식의 차이입니다.

Epic 공식 자료

[05] Programming with Substrate GBuffer Formats
[06] Decal Materials
[10] Overview of Substrate Materials

12 Translucency와 굴절

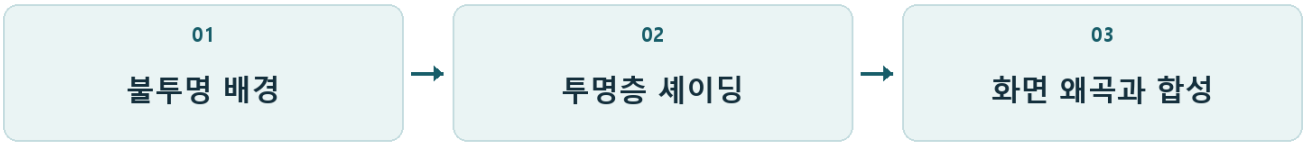


그림 12 / Translucency와 굴절 개념도

원리 반투명은 배경이 보이는 표면의 색과 뒤의 색을 합성한다. 단순 알파 합성에서 결과는 앞색×α + 뒤 색×(1 α)다. 굴절은 빛 방향 변화로 배경이 휘어 보이는 효과다.

작동 과정 많은 UE 반투명 경로는 불투명 처리 뒤 별도 조명 방식과 정렬을 사용한다. 반투명 메시에 설정한 Sort Priority는 큰 순서를 조절하지만 서로 교차하는 면의 모든 픽셀 순서를 해결할 수 없다. 화면 기반 굴절은 현재 프레임에 없는 정보에서 제한된다.

아트 실무 쇼윈도의 유리, 김 서림, 빗방울을 중첩하기 전에 한 표면 안에서 표현 가능한 부분을 찾아본다. 화면 가장자리와 배경이 얇은 프레임을 지날 때 굴절 깨짐을 확인한다.

성능 비용 겹치는 레이어, 화면 커버리지, 조명 모드, 굴절과 후처리 합성이 관여한다. 단순한 그래프도 큰 투명 레이어를 여러 장 쌓으면 비용이 커질 수 있다.

언리얼 실습 ① 동일 유리판을 1장과 4장으로 배치한다. ② 굴절을 끈 재질과 켠 재질을 비교한다. ③ 카메라를 옆으로 이동해 정렬과 가장자리 깨짐을 확인하고 Translucency 관련 GPU 그룹을 기록한다.

핵심 요약 반투명은 레이어별로 그리기와 합성이 필요하므로 화면 면적을 기준으로 검증한다.

Epic 공식 자료

[13] Lit Translucency
[14] Using Refraction

13 SSS와 표면 아래 산란

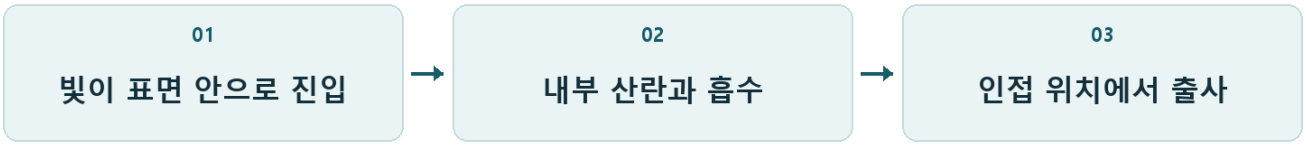


그림 13 / SSS와 표면 아래 산란 개념도

원리 Subsurface Scattering은 빛이 물질 내부로 들어가 산란한 뒤 다른 위치에서 다시 나오는 효과다. 왁스나 대리석의 빛 번짐을 설명한다.

작동 과정 UE의 Subsurface Profile은 화면 공간에서 확산을 근사한다. Substrate의 SSS MFP는 매질 내부에서 상호작용하기까지의 평균 거리 스케일이며 색 채널별 특성을 줄 수 있다. 모델, 표면 두께와 조명에 따라 결과가 달라진다.

아트 실무 조각상과 촛농에서 두꺼운 부분은 밀도감을 유지하고 가장자리나 얇은 부분에서 역광을 확인한다. 식생의 Two Sided Foliage와 왁스의 확산 프로파일은 서로 다른 표현 목적에 맞춘다.

성능 비용 화면 공간 산란이나 복잡한 투과 평가에는 추가 GPU 작업이 들 수 있다. 화면에서 차지하는 면적, 프로파일과 구현 경로를 함께 기록한다.

언리얼 실습 ① 실측 스케일의 왁스 기둥과 얇은 판을 만든다. ② Default Lit와 Subsurface Profile 또는 Substrate SSS 버전을 분리한다. ③ 같은 노출에서 전면·역광을 비교하고 재질이 화면을 크게 채울 때의 GPU 그룹도 기록한다.

핵심 요약 SSS의 관찰 포인트는 물체 뒤 배경이 아니라 재질 내부를 지난 빛의 확산이다.

Epic 공식 자료

[10] Overview of Substrate Materials
[15] Subsurface Profile Shading Model

14 Draw Call과 배칭

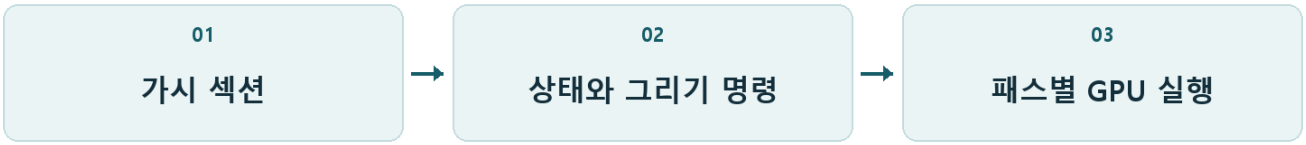


그림 14 / Draw Call과 배칭 개념도

원리 Draw Call은 GPU에 지오메트리를 그리도록 제출하는 명령이다. Actor 하나에 메시 섹션이 여럿이면 패스마다 여러 작업이 생길 수 있다.

작동 과정 렌더러는 가시성, 메시 섹션, 머티리얼과 파이프라인 상태를 바탕으로 그리기 명령을 준비한다. 호환되는 데이터는 엔진의 배칭이나 인스턴싱으로 합쳐질 수 있다. 섀도와 깊이 패스의 수치는 메인 패스와 다르다.

아트 실무 골목 소품 하나에 금속, 나무, 흙 슬롯을 쓰는 대신 텍스처 아틀라스와 공유 머티리얼을 후보로 둔다. 동네 전체를 하나로 합치면 부분 가림의 이점을 잃을 수 있다.

성능 비용 제출과 상태 준비는 CPU 부담이 될 수 있다. 호출이 줄어도 화면 픽셀, 셰이더, 그림자 작업이 유지되면 GPU 시간 변화는 작을 수 있다.

언리얼 실습 ① 같은 형상의 3섹션과 1섹션 메시를 준비한다. ② 카메라와 배치 수를 맞춘다. ③ stat scenerendering 또는 stat rhi의 관련 카운터와 stat unit의 Draw를 기록한다. GPU 캡처로 어떤 패스에서 작업이 줄었는지 확인한다.

핵심 요약 Actor 수를 Draw Call 수로 환산하지 않고 패스와 섹션을 실제로 확인한다.

Epic 공식 자료

[03] Introduction to Performance Profiling and Configuration
[16] Mesh Drawing Pipeline

15 Instancing과 반복 배치

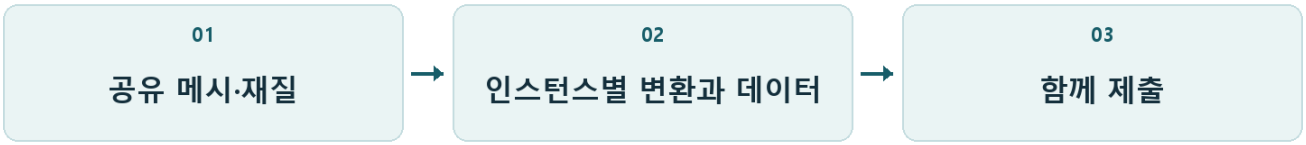


그림 15 / Instancing과 반복 배치 개념도

원리 인스턴싱은 같은 메시 리소스를 여러 배치에서 재사용한다. ISM 및 HISM은 다수 반복 오브젝트를 컴포넌트 단위로 구성하는 대표 방법이다.

작동 과정 메시와 머티리얼을 공유하고 인스턴스마다 변환 및 선택적으로 Per Instance Custom Data를 제공한다. HISM은 계층을 구성해 인스턴스 그룹 처리를 지원한다. 가시성·LOD 세부 동작은 기능과 버전에 따라 달라진다.

아트 실무 울타리 기둥 200개를 반복 배치하고 Per Instance Custom Data로 밝기나 색을 바꾼다. 메시 슬롯과 그림자 조건은 일치시킨다.

성능 비용 Actor·컴포넌트와 제출 부담을 낮출 여지가 있지만 인스턴스별 지오메트리와 픽셀 작업은 발생한다. 자주 갱신되는 변환, 군집의 범위와 가시성도 비용에 영향을 준다.

언리얼 실습 ① Static Mesh Actor 200개와 ISM 200개를 서로 다른 레벨 변형에 둔다. ② 전부 보이는 위치와 일부만 보이는 위치에서 Draw와 GPU를 기록한다. ③ 같은 메시의 HISM 버전도 필요 시 비교하고 충돌·그림자 조건을 맞춘다.

핵심 요약 공유 리소스로 제출과 관리 비용을 줄이고 실제 가시 작업량은 따로 잰다.

Epic 공식 자료

[16] Mesh Drawing Pipeline
[17] Instanced Static Mesh Component

16 LOD와 화면 크기

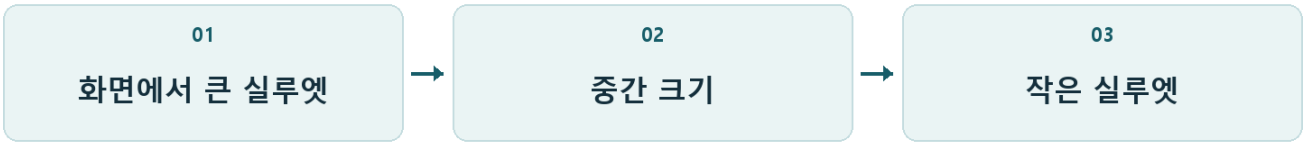


그림 16 / LOD와 화면 크기 개념도

원리 LOD는 화면에서 필요로 하는 디테일에 맞춰 대체 표현을 선택하는 방식이다. 전통 Static Mesh LOD는 화면 크기 기준으로 작성할 수 있다.

작동 과정 LOD0에서 먼 단계로 갈수록 삼각형, 섹션 또는 재질 복잡도를 낮출 수 있다. 기준이 화면 크기이므로 오브젝트 스케일과 카메라 FOV에 따라 같은 거리에서도 선택이 달라진다.

아트 실무 난간의 나사와 작은 베벨은 먼저 줄이고, 위쪽 윤곽과 굵은 기둥 간격은 유지한다. 색, 노멀, 그림자 실루엣이 전환 시 튀지 않는지 본다.

성능 비용 정점 처리와 그림자 지오메트리를 줄일 수 있다. 섹션 수가 같고 화면을 덮는 면적이 비슷하면 호출과 픽셀 비용은 여전히 남는다. 디더 전환 구간도 측정한다.

언리얼 실습 ① Static Mesh Editor에서 LOD별 화면 크기와 삼각형 수를 확인한다. ② 가능하면 LOD Coloration 또는 Forced LOD로 개별 형태를 본다. ③ 고정 이동 경로에서 실루엣·그림자 튐과 GPU ms를 비교한다.

핵심 요약 디테일 삭제의 기준은 월드 거리보다 실제 화면에서 읽히는 크기다.

Epic 공식 자료

[17] Instanced Static Mesh Component
[18] Optimizing LOD Screen Size Per Platform

17 Nanite와 가상화 지오메트리

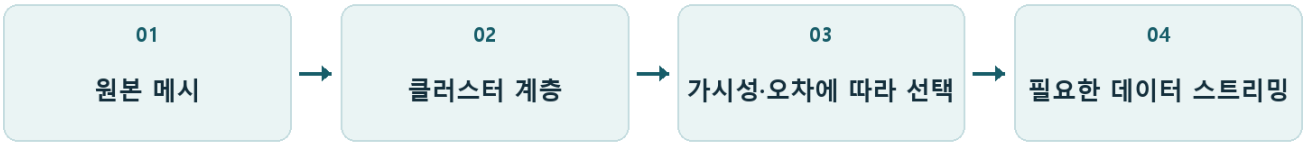


그림 17 / Nanite와 가상화 지오메트리 개념도

원리 Nanite는 메시 데이터를 계층적 클러스터로 구성해 필요한 디테일을 선택하고 스트리밍하는 가상화 지오메트리 시스템이다.

작동 과정 화면에서 필요한 오차 수준과 가시성에 따라 클러스터를 고른다. Nanite의 전용 래스터·가시성 경로는 전통 메시의 그리기 방식과 다르다. 폴백 메시와 다른 기능의 연결도 대상 플랫폼별로 확인한다.

아트 실무 절벽처럼 복잡한 고밀도 표면에 적용한 뒤, 얇은 잎을 여러 겹 쌓은 수관도 따로 시험한다. 가까운 표면들이 깊이상 겹치면 Nanite에서도 작업량이 늘 수 있다.

성능 비용 높은 원본 폴리곤을 다룰 수 있어도 인스턴스 수, 클러스터 가시성, Masked/WPO 재질, 화면 겹침, 디스크와 스트리밍 자원은 남는다. 적용 여부로 조명과 그림자 경로도 달라질 수 있다.

언리얼 실습 ① 같은 절벽 배치에서 Nanite와 전통 LOD를 비교한다. ② Nanite Visualization의 Triangles, Clusters, Overdraw를 확인한다. ③ 근거리·먼거리와 빠른 이동에서 GPU, 팝인, 그림자를 기록한다. 폴백 품질이 비교를 왜곡하는지도 확인한다.

핵심 요약 Nanite는 지오메트리 선택을 바꾸며 화면 겹침과 재질 비용은 계속 측정해야 한다.

Epic 공식 자료

[17] Instanced Static Mesh Component
[19] Nanite Virtualized Geometry

18 Culling과 가시성

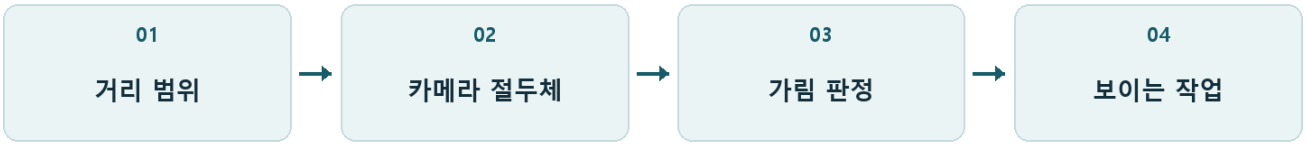


그림 18 / Culling과 가시성 개념도

원리 Culling은 렌더링에 참여할 후보를 줄이는 과정이다. 거리, 절두체, 가림은 서로 다른 기준을 사용한다.

작동 과정 엔진은 보통 오브젝트 bounds를 사용해 가시성을 판정한다. 카메라 밖의 후보를 빼고 가림 판정에는 깊이 기반 방법 등을 쓴다. 메인 뷰에서 컬링된 물체도 그림자 등 별도 뷰에서 영향을 줄 수 있다.

아트 실무 건물 내부까지 포함한 긴 합체 메시가 벽 뒤에서도 보이는 것으로 남으면 공간 단위 분할을 검토한다. WPO가 큰 풀은 너무 작은 bounds에서 사라지는지 본다.

성능 비용 분할이 세밀하면 가림 효율은 좋아질 수 있지만 컴포넌트, 가시성 테스트, 제출 부담이 늘 수 있다. 큰 bounds는 판정 기회를 줄인다.

언리얼 실습 ① 꺾인 복도에서 합체 벽과 구획별 벽을 비교한다. ② Viewport의 Bounds를 켜고 지원 경로에서 r.VisualizeOccludedPrimitives 1을 확인한다. ③ 카메라를 모서리 너머로 움직여 가시 수치와 팝인을 비교한다.

핵심 요약 좋은 분할 단위는 제작 모듈과 렌더링 가시성의 균형으로 정한다.

Epic 공식 자료

[17] Instanced Static Mesh Component
[19] Nanite Virtualized Geometry
[30] Visibility and Occlusion Culling

19 Mipmap과 텍스처 샘플링

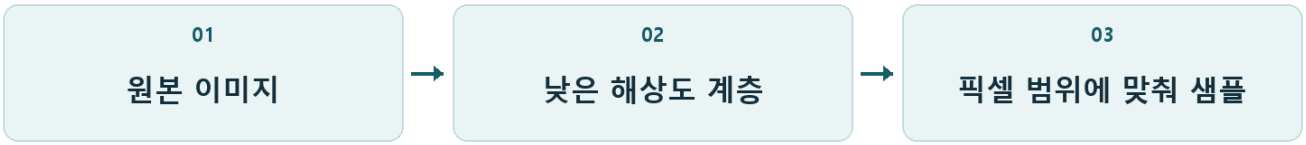


그림 19 / Mipmap과 텍스처 샘플링 개념도

원리 Mip은 텍스처의 축소 계층이다. 샘플러는 화면 픽셀에 해당하는 UV 변화량을 바탕으로 적절한 해상도를 고른다.

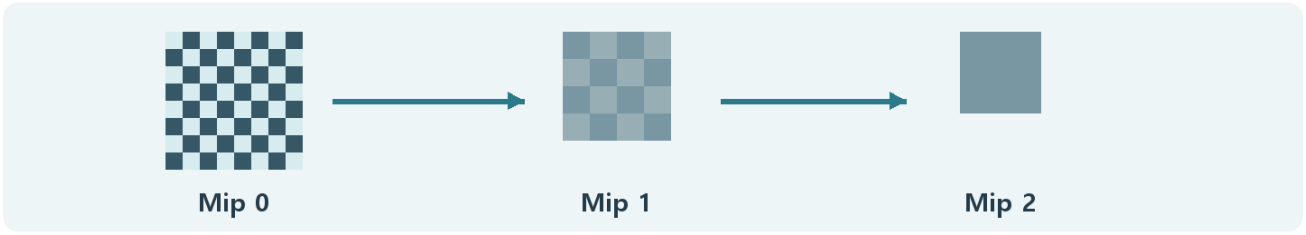
작동 과정 원거리나 비스듬한 표면에서 한 픽셀이 넓은 텍스처 영역을 덮는다. 축소 계층과 필터링이 고주파 패턴의 깜빡임을 줄인다. 이방성 필터링은 길게 늘어난 픽셀 범위에 대응한다.

아트 실무 타일이 많은 복도 바닥을 낮은 시선에서 본다. 원거리에서 줄눈이 반짝이면 밉과 필터, 텍스처·노멀의 고주파 패턴을 살핀다.

성능 비용 완전한 2D 밉 체인은 이상적인 정사각형에서 최상위 밉보다 약 1/3의 텍셀을 더 갖는다. 실제 저장량은 압축과 블록 배치에 따라 달라지고, 낮은 밉 선택은 캐시와 대역폭에 유리할 수 있다.

언리얼 실습 ① 원본 텍스처를 복제해 일반 밉과 NoMipmaps를 비교한다. ② Texture Editor에서 개별 밉과 데이터 맵 색 공간을 확인한다. ③ 동일 카메라 이동에서 바닥의 시간적 깜빡임을 기록한다. 스트리밍 상태도 같이 확인한다.

핵심 요약 밉은 샘플링 품질 도구이며 어떤 밉을 메모리에 유지할지는 스트리밍이 다룬다.



샘플링 개념도 / 축소 밉은 넓은 영역의 패턴을 평균화합니다.

Epic 공식 자료

[20] Texture Asset Editor

[21] Texture Streaming Overview

20 Texture Streaming과 메모리 예산

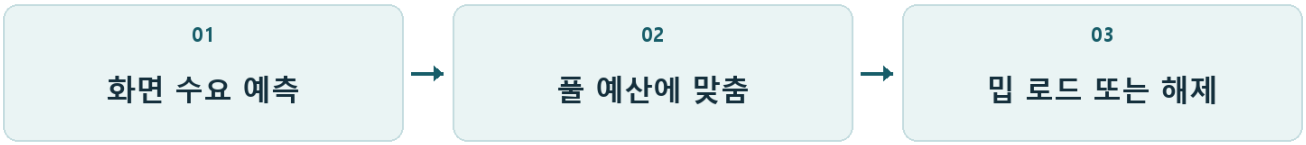


그림 20 / Texture Streaming과 메모리 예산 개념도

원리 일반 Texture Streaming은 텍스처별 필요한 밉을 판단해 메모리에 상주시킨다. Virtual Texture의 타일 스트리밍 및 월드 액터 스트리밍과 별개의 시스템이다.

작동 과정 스트리머는 뷰, 거리와 텍셀 밀도 등을 사용해 원하는 해상도를 추정한다. 메모리 예산과 우선순위에 따라 로드·해제를 결정하고 I/O를 요청한다. 원하는 밉과 상주 밉이 잠시 다를 수 있다.

아트 실무 시장 전경에 4K 소품이 몰려 있으면 근접 관찰이 가능한 간판과 먼 배경 용기를 분리해 해상도를 정한다. 흐린 재질은 UV 스케일과 스트리밍 데이터도 점검한다.

성능 비용 큰 상주 밉은 메모리와 전송량을 늘리고 급격한 이동은 로딩 지연을 드러낼 수 있다. 풀을 키우면 다른 리소스에 쓸 메모리가 줄어든다.

언리얼 실습 ① 빌드에서 stat streaming을 켠다. ② 같은 카메라로 콜드 진입과 웜 재방문을 비교한다. ③ 일부 소품의 최대 해상도만 낮추고 풀 압박, 질감 선명도와 팝인을 비교한다. 변경 전후 스트리밍 데이터도 갱신한다.

핵심 요약 풀 부족, 잘못 추정된 수요, I/O 지연을 분리해 진단한다.

Epic 공식 자료

[20] Texture Asset Editor
[21] Texture Streaming Overview
[22] Texture Streaming Metrics

21 Lumen과 동적 간접광

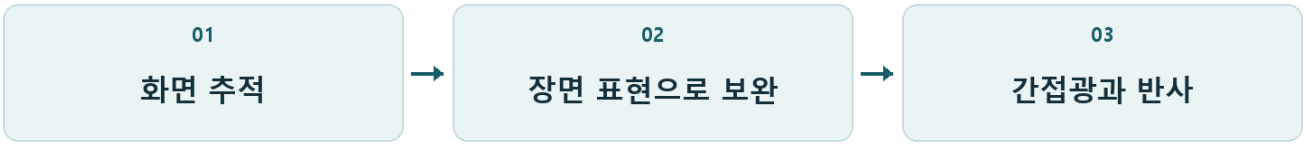


그림 21 / Lumen과 동적 간접광 개념도

원리 Lumen은 동적 전역 조명과 반사를 계산한다. 빨간 벽에서 튄 빛이 흰 벽에 색을 입히는 현상은 간접광의 대표 사례다.

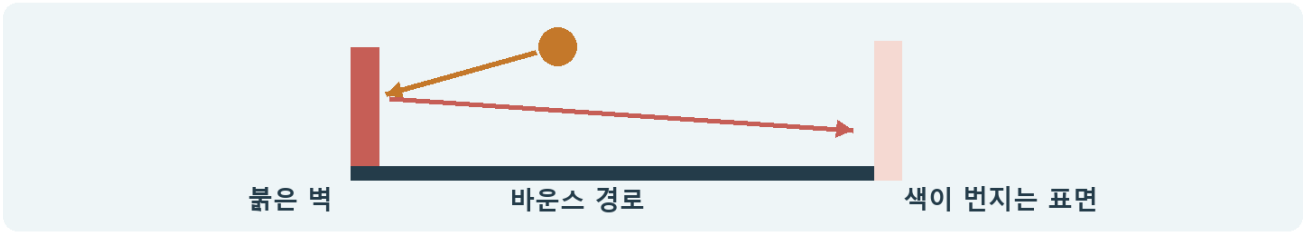
작동 과정 화면 추적을 먼저 활용하고 화면에서 해결하기 어려운 부분을 소프트웨어 거리장 또는 하드웨어 레이 트레이싱으로 보완한다. Surface Cache는 표면 정보를 여러 방향에서 캡처해 조회에 사용한다. 반사 설정에 따라 히트 위치에서 조명을 평가하는 경로도 있다.

아트 실무 한 메시로 만든 방에서 벽이 검게 반사되면 Surface Cache의 분홍 영역을 본다. 벽, 바닥, 천장을 분리한 변형을 만들고 얇은 벽의 누광은 거리장 표현도 살핀다.

성능 비용 내부 해상도, 추적 방식과 거리, 반사 범위, 장면 표현 갱신이 비용을 바꾼다. HWRT는 지원 GPU를 요구하며 인스턴스 수와 변형이 큰 장면에서는 가속 구조 갱신도 중요하다.

언리얼 실습 ① 간접광이 보이는 L자 방을 만들고 노출을 고정한다. ② Lumen Overview, Lumen Scene, Surface Cache를 확인한다. ③ 메시 분할 또는 추적 설정 한 가지만 바꾸고 간접광·반사 품질과 GPU 그룹을 비교한다.

핵심 요약 반사·간접광 문제를 화면 정보 부족, 장면 표현 누락, 계산 예산으로 나눠 확인한다.



조명 개념도 / 입사광이 붉은 벽에서 반사되어 맞은편에 영향을 줍니다.

Epic 공식 자료

[23] Lumen Technical Details

[24] Lumen Performance Guide

22 Virtual Shadow Maps와 페이지 캐시

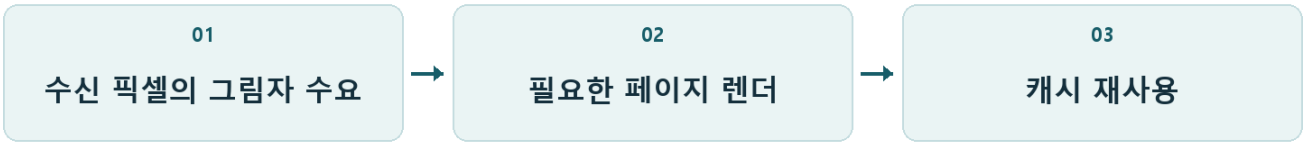


그림 22 / Virtual Shadow Maps와 페이지 캐시 개념도

원리 VSM은 고해상도 가상 그림자 맵을 페이지 단위로 제공한다. 방향광은 카메라 주변 해상도를 효율적으로 배치하기 위해 클립맵을 사용한다.

작동 과정 보이는 표면이 요구하는 페이지를 할당하고 라이트 관점에서 가림을 기록한다. 이전 프레임의 페이지를 재사용하지만 움직이는 라이트나 그림자 캐스터, WPO 등이 캐시 무효화를 일으킬 수 있다.

아트 실무 바람에 흔들리는 풀밭이 멀리까지 그림자를 만들면 원거리 WPO와 그림자 표현을 각각 비교한다. 라이트 여러 개가 같은 수관을 비추면 영향 반경도 검토한다.

성능 비용 페이지 갱신과 그림자 투영은 별도 비용이다. 새로 드러난 페이지, 큰 변형 bounds, 겹치는 그림자 라이트는 시간과 페이지 풀 압박에 영향을 준다.

언리얼 실습 ① Shadow Map Method에서 Virtual Shadow Maps를 확인한다. ② 고정 카메라에서 VSM Cached Page 시각화로 바람 켜짐·꺼짐을 비교한다. ③ 같은 조건으로 카메라를 움직여 Shadow Depths와 Projection 그룹을 나눠 기록한다.

핵심 요약 VSM 비용은 그림자 선명도와 함께 페이지의 재사용률과 갱신 범위를 본다.

Epic 공식 자료

[19] Nanite Virtualized Geometry
[25] Virtual Shadow Maps

23 TAA와 프레임 히스토리

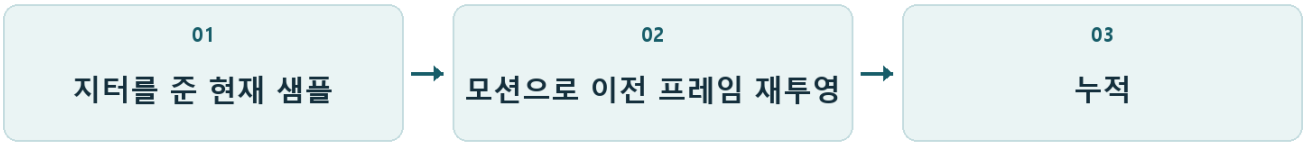


그림 23 / TAA와 프레임 히스토리 개념도

원리 Temporal AA는 여러 프레임의 서브픽셀 정보를 활용해 계단과 깜빡임을 줄인다.

작동 과정 카메라 투영에 작은 지터를 주어 다른 위치를 샘플한다. 모션 벡터로 히스토리를 현재 화면에 맞추고 새로 드러난 표면 등 신뢰하기 어려운 정보는 제한한다. 누적의 세부 방식은 구현에 따라 달라진다.

아트 실무 가느다란 철망과 WPO 식생은 정지 화면에서 매끄러워도 이동하면 번지거나 자국을 남길 수 있다. 불투명 물체와 투명 효과의 모션 정보는 개별적으로 확인한다.

성능 비용 히스토리 버퍼와 재투영 패스가 필요하다. 시간적 안정성을 높이는 설정은 움직임 중 선명도와 응답성에 영향을 줄 수 있다.

언리얼 실습 ① Project Settings에서 Temporal AA를 선택하고 100% 내부 해상도로 시작한다. ② 철망 앞을 일정 속도로 이동하고 정지 장면도 캡처한다. ③ WPO 켜짐·꺼짐에서 Velocity 시각화가 지원되면 비교하고 고스팅과 깜빡임을 기록한다.

핵심 요약 TAA 품질은 정지 스크린샷과 이동 영상 둘 다에서 판단한다.

Epic 공식 자료

[26] Anti-Aliasing and Upscaling

[27] Temporal Super Resolution

24 TSR과 시간적 업스케일링

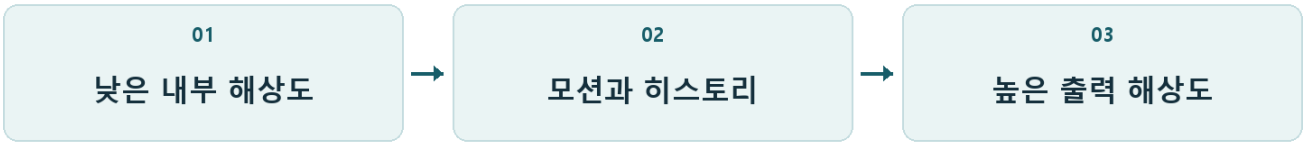


그림 24 / TSR과 시간적 업스케일링 개념도

원리 Temporal Super Resolution은 내부 해상도의 프레임과 히스토리로 더 높은 출력 해상도의 영상을 재구성하는 UE 시간적 업스케일러다.

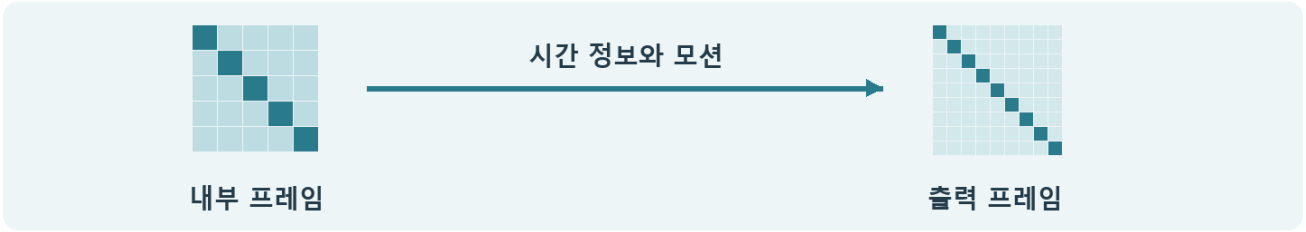
작동 과정 낮은 해상도로 장면을 그리고 지터·깊이·모션과 이전 결과를 사용해 더 촘촘한 출력 이미지를 만든다. 드러난 표면과 움직임을 판별해 이전 샘플의 기여를 조절한다.

아트 실무 1440p 출력으로 골목을 달리며 젖은 벽 반사와 간판 글자, 얇은 케이블이 유지되는지 본다. 화면 정지 후에만 선명해진다면 이동 중의 품질도 기준에 넣는다.

성능 비용 출력 2560×1440에서 67% 내부 해상도의 픽셀 수는 약 45%다. 그에 비례해 줄어드는 것은 픽셀 의존 작업의 일부다. TSR 자체의 히스토리와 출력 처리가 들고 CPU와 일부 그림자 비용은 별개다.

언리얼 실습 ① Anti-Aliasing Method를 TSR로 설정한다. ② 출력 해상도와 동적 해상도 상태를 고정하고 r.ScreenPercentage 100과 67을 비교한다. ③ 동일 이동에서 TSR 및 주요 패스 ms와 글자·철망 품질을 기록한다.

핵심 요약 화질 판정은 같은 출력 해상도에서, 성능 판정은 절약한 작업과 재구성 비용을 함께 본다.



격자 개념도 / 시간적 재구성이 내부 프레임을 더 촘촘한 출력에 대응시킵니다.

Epic 공식 자료

[26] Anti-Aliasing and Upscaling
[27] Temporal Super Resolution

25 CPU와 GPU 병목

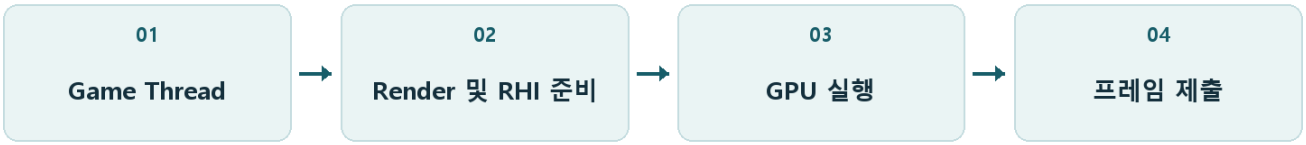


그림 25 / CPU와 GPU 병목 개념도

원리 병목은 해당 구간에서 프레임 처리율을 제한하는 작업이다. CPU와 GPU는 다른 프레임을 겹쳐 처리할 수 있어 각 줄의 시간을 단순 합산하면 프레임 시간과 맞지 않는다.

작동 과정 stat unit에서 Frame, Game, Draw, GPU, 지원 시 RHIT를 확인한다. 가장 큰 값은 출발점이며 스레드 대기, VSync, 프레임 제한, GPU 큐 지연으로 비슷한 수치가 나올 수 있다.

아트 실무 도심의 화면을 아래로 돌려 GPU ms만 급락하면 화면 픽셀과 조명 작업을 조사한다. 화면 해상도를 줄여도 프레임이 그대로면 CPU 제출, 게임 로직, 동기화와 제한을 본다.

성능 비용 60 FPS는 약 16.67 ms, 30 FPS는 약 33.33 ms의 프레임 간격을 요구한다. 평균만으로 순간 끊김을 알기 어려우므로 프레임 시간 상위 백분위와 최대 구간을 같이 본다.

언리얼 실습 ① 목표 기기의 Development 빌드에서 stat unit과 stat unitgraph를 기록한다. ② GPU 진단용으로 내부 해상도만 낮춰 반응을 본다. ③ Unreal Insights에서 병목 프레임의 CPU 스레드와 GPU 타임라인, 대기 구간을 확인한다.

핵심 요약 프레임 제한과 대기를 확인한 뒤 실제로 처리율을 제한하는 작업을 고른다.

Epic 공식 자료

[03] Introduction to Performance Profiling and Configuration

[28] Stat Commands

26 GPU Profiler와 전후 검증

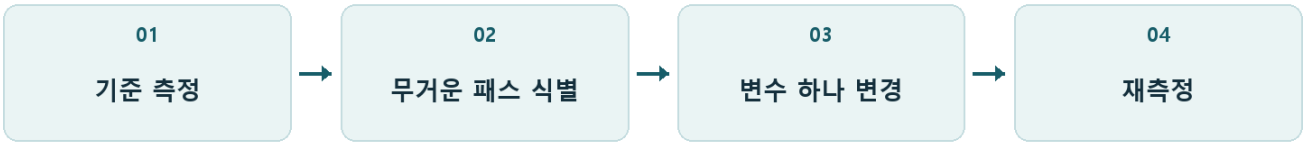


그림 26 / GPU Profiler와 전후 검증 개념도

원리 GPU 프로파일링은 프레임의 GPU 이벤트를 시간 구간으로 나눠 어떤 작업에 비용이 쓰였는지 보는 과정이다. 단일 캡처와 반복 측정의 역할이 다르다.

작동 과정 stat gpu로 큰 그룹을 본 뒤 profilegpu 또는 Unreal Insights의 GPU 트랙으로 상세 이벤트를 확인한다. UE 5.6 이후 새 GPU Profiler가 기본인 구성에서는 구형 GPU Visualizer 팝업을 전제로 하지 않고 로그와 Insights를 본다. 중첩 이벤트와 비동기 큐 시간을 단순 합산하지 않는다.

아트 실무 가상의 골목 A/B에서 데칼 범위만 조정한다. 예를 들어 Decals가 1.3→0.7 ms, 전체 GPU가 17.2→16.7 ms라면 각각 0.6 ms와 0.5 ms 개선이다. CPU가 19 ms라면 프레임 처리율 개선 여부는 따로 확인한다.

성능 비용 캡처와 에디터 자체에도 오버헤드가 있다. 셰이더 컴파일·스트리밍의 초기 스파이크와 안정 구간을 분리하고 목표 기기·설정에서 반복한다.

언리얼 실습 ① 동일 카메라 경로·출력·스케일·설정을 기록하고 워밍업한다. ② 3회씩 같은 길이로 실행해 Frame 중앙값과 P95, GPU 중앙값과 해당 패스를 기록한다. ③ 패스 계층을 확인하고 움직임 영상의 품질을 비교한다. ④ 되돌려 개선이 재현되는지 본다.

핵심 요약 패스 시간, 총 프레임 시간, 이미지 품질을 같은 조건의 전후 기록으로 남긴다.

Epic 공식 자료

[03] Introduction to Performance Profiling and Configuration
[28] Stat Commands
[29] Beyond Print String · GPU Profiler

골목 씬의 성능 분석

재현 환경

모퉁이를 도는 30초 카메라 경로를 저장합니다. 벽의 데칼, 가로등, 유리, 풀을 포함합니다. 장치·드라이버·UE 버전·RHI·빌드 구성·출력과 내부 해상도·Scalability를 적습니다. 프레임 제한과 VSync는 진단에 맞게 통제하고 제품 설정에서도 다시 확인합니다.

첫 번째 관찰

모퉁이 진입 순간만 끊기면 웜 재방문과 비교합니다. 첫 진입에서만 반복되면 스트리밍이나 PSO·셰이더 준비를 조사합니다. 항상 같은 프레임에서 GPU가 길다면 GPU 트랙에서 무거운 패스를 확인합니다.

두 번째 관찰

풀밭을 보는 동안 Shadow Depths가 길면 VSM 페이지 시각화를 확인합니다. 원거리 WPO만 끈 A/B와 그림자만 바꾼 A/B를 따로 만든 뒤 그림자 움직임과 시각적 차이를 기록합니다.

기록할 항목

변경 설명 / 카메라 구간 / 테스트 횟수 / Frame 중앙값과 P95 / Game 및 Draw / GPU 중앙값 / 패스와 ms / 스트리밍 피크 / 이동 영상의 품질 차이 / 재현 여부. P95가 높으면 가장 느린 5% 부근의 구간을 타임라인에서 조사합니다.

결과 해석

패스의 시간이 줄어도 다른 병목으로 전체 시간이 유지될 수 있습니다. 전체 시간과 해당 패스가 함께 줄고 품질이 허용 범위라면 변경의 효과를 기록할 수 있습니다. 비용이 다른 패스로 옮겨갔다면 그 변화도 함께 남깁니다.

이전 설명에서 구분한 항목

Depth는 별도 리소스입니다. 고정 GBuffer 포맷에서 그래프 노드가 늘어도 저장 바이트는 그대로일 수 있습니다. DBuffer 데칼과 GBuffer 직접 합성은 시점이 다릅니다. Adaptive의 closure 예산, Blendable의 60Hz 지향, 런타임 프레임 시간은 별개입니다. UE 5.6 이후 GPU 프로파일링의 출력 위치도 구형 Visualizer와 다를 수 있습니다.

출처 목록

Epic Games 문서 및 공식 발표 / UE 5.8 문서 기준

01 Rendering Settings

02 Render Dependency Graph

03 Introduction to Performance Profiling and Configuration

04 Forward Shading Renderer

05 Programming with Substrate GBuffer Formats

06 Decal Materials

07 Viewport Modes

08 Physically Based Materials

09 Using Fresnel

10 Overview of Substrate Materials

11 Material Editor UI

12 Material Inputs

13 Lit Translucency

14 Using Refraction

15 Subsurface Profile Shading Model

출처 목록 계속

Epic Games 문서 및 공식 발표 / UE 5.8 문서 기준

16 Mesh Drawing Pipeline

17 Instanced Static Mesh Component

18 Optimizing LOD Screen Size Per Platform

19 Nanite Virtualized Geometry

20 Texture Asset Editor

21 Texture Streaming Overview

22 Texture Streaming Metrics

23 Lumen Technical Details

24 Lumen Performance Guide

25 Virtual Shadow Maps

26 Anti-Aliasing and Upscaling

27 Temporal Super Resolution

28 Stat Commands

29 Beyond Print String · GPU Profiler

30 Visibility and Occlusion Culling

31 Real Shading in Unreal Engine 4 · Brian Karis

32 Scripting the Editor Using Python